OPTIMISASI ALGORITMA KOMPRESI LEMPEL-ZIV-WELCH DENGAN TRANSFORMASI BERKAS BURROWS WHEELER

Laporan Tugas Akhir

Oleh

JONATHAN MARCEL T

NIM: 13507072

Disusun sebagai syarat kelulusan tingkat sarjana



PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
Juli 2011

OPTIMISASI ALGORITMA KOMPRESI LEMPEL-ZIV-WELCH DENGAN TRANSFORMASI BERKAS BURROWS WHEELER

Laporan Tugas Akhir

Oleh

JONATHAN MARCEL T

NIM: 13507072

Telah disetujui dan disahkan sebagai laporan tugas akhir
di Bandung, pada tanggal 24 Juni 2011

Pembimbing

Dra. Harlili, M.Sc.
NIP. 195711231984032001

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO & INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

ABSTRAK

OPTIMISASI ALGORITMA KOMPRESI LEMPEL-ZIV-WELCH DENGAN TRANSFORMASI BERKAS BURROWS WHEELER

oleh

JONATHAN MARCEL T

Jumlah data dan informasi yang disimpan manusia mengalami keterbatasan karena kecepatan pertumbuhan jumlah data jauh lebih besar dibandingkan pertumbuhan kapasitas media penyimpanan. Oleh karena itu, diciptakanlah berbagai algoritma kompresi data, sehingga dengan kapasitas media penyimpanan yang sama manusia dapat menyimpan data dengan jumlah yang lebih banyak. Salah satu algoritma kompresi data yang terkenal adalah algoritma Lempel-Ziv-Welch (LZW).

Algoritma LZW memiliki kelemahan antara lain manajemen kamus yang semakin rumit apabila ukuran berkas yang akan dimampatkan semakin besar. Oleh karena itu, kelemahan tersebut akan dicoba diatasi dengan transformasi Burrows Wheeler. Transformasi Burrows Wheeler menggunakan sistem permutasi pada setiap karakter di dalam berkas. Dengan transformasi ini diharapkan karakter-karakter dalam berkas akan menjadi berdekatan sehingga pada saat proses kompresi, berkas yang dihasilkan akan lebih baik rasio kompresinya dibandingkan berkas yang belum ditransformasi.

Optimisasi algoritma kompresi LZW dilakukan dengan melakukan transformasi Burrows Wheeler pada berkas yang akan dikompresi, dengan kata lain transformasi ini menjadi *pre-processing* berkas yang akan dikompresi. Implementasi dan pengujian kinerja optimisasi dilakukan dengan menggunakan bahasa C# . Perangkat lunak ini dapat mengkompresi berkas berbentuk *plain text* dengan algoritma LZW maupun gabungan algoritma LZW dan Burrows Wheeler.Ukuran kinerja yang digunakan adalah rasio kompresi.

Data uji yang digunakan adalah *calgary corpus*, *canterbury corpus*, teks biasa, dan berkas ekstrim. Berdasarkan hasil pengujian, ternyata algoritma LZW sendiri mampu mengkompresi berkas menjadi 40%-60% dari berkas awal. Setelah dioptimisasi, ternyata transformasi Burrows Wheeler dapat meningkatkan kinerja algoritma kompresi LZW. Secara kuantitatif, kinerja yang ditingkatkan berkisar antara 0,3%-30% bergantung kepada jenis berkas yang dikompresi.

Kata Kunci: Lempel-Ziv-Welch, Burrows Wheeler, kompresi data

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Tuhan Yesus Kristus, atas bimbingan-Nya dan berkat-Nya sehingga tugas akhir yang berjudul “Optimisasi Algoritma Kompresi Lempel-Ziv-Welch dengan Transformasi Berkas Burrows Wheeler” dapat diselesaikan dengan baik.

Selama masa pengerjaan tugas akhir ini, penulis mendapat banyak dukungan dari berbagai pihak. Oleh karena itu, penulis ingin mengucapkan terimakasih sebesar-besarnya kepada:

1. Papa dan Mama.Tanpa dukungan, doa, dan kasih sayang dari kalian, penulis tidak akan berhasil menyelesaikan tugas akhir ini dengan tepat waktu. Dan juga kepada Koko yang telah banyak mendukung penulis selama bertempat tinggal di Bandung.
2. Ibu Harlili, selaku dosen pembimbing yang senantiasa memberikan dukungan, bimbingan, dan masukan-masukan serta menyemangati penulis agar tepat waktu untuk menyelesaikan tugas akhir.
3. Ibu Masayu dan Pak Rinaldi, selaku dosen penguji pada saat sidang tugas akhir, yang telah memberikan banyak masukan untuk penyempurnaan tugas akhir ini.
4. Teman-teman baik penulis, khususnya teman-teman dari St. Lucia Choir, antara lain Inge, Erwin, Alex, Christ, Arif, Tandrey, Darryl, Maxi, Kevin Satopoh, Kevin Pratama, Ratna, Cindy, Ami, Amel, Rina, Janice, Renata, Vina, Tasia, dan teman-teman lainnya. Terima kasih atas masa-masa terindah selama penulis bergabung menjadi St. Lucia Choir. Terima kasih juga karena kalian telah menjadi sahabat yang baik dalam suka maupun duka.
5. Teman-teman yang sering berkumpul di laboratorium Basis Data, khususnya Firdi Mulia dan Aris Feryanto, yang telah banyak sekali membantu dalam mengerjakan tugas akhir ini. Juga kepada Hendy, Kevin, James, Adiputra, Tommy, Dito, Galih, dan lain-lain, terima kasih karena telah menjadi teman yang baik, dalam kegiatan akademis maupun non-akademis.

6. Teman-teman himpunan informatika, khususnya teman-teman terdekat di informatika 2007 yaitu Calvin, Amalfi, Raditya, Dimas, Dannis, Budi, Ramos, Trendyar, Afif, Prawidar, Deto, Sanrio, Shauma, Edri, Riris, Elvina, Bhella, Halida, Dellyna, dan lain-lain. Terima kasih atas kehadiran kalian dalam hidup penulis. Kalian adalah salah satu teman-teman terbaik yang pernah penulis miliki.
7. Segenap staff di Grha Ciumbuleuit yang telah mendukung penulis selama bertempat tinggal di Bandung.
8. Ciangsem Aping, Ciangsuk Wei, Ryan, dan Renata. Kalian seperti keluarga kedua penulis selama berada di Bandung. Terima kasih atas dukungan dan bantuannya. Semoga keluarga kalian tetap berbahagia sampai selama-lamanya.
9. Teman-teman baik yang sama-sama berasal dari Manado, yaitu Rendy, Yesq, Edwin. Kalian adalah sahabat terbaik saya sejak penulis masih kecil.
10. Seluruh dosen yang pernah mengajari berbagai macam hal kepada penulis selama penulis berkuliah di ITB. Terima kasih atas jasa-jasamu terhadap penulis. Tidak ada yang dapat membalas pengabdian kalian selain ucapan terima kasih yang sebesar-besarnya dari penulis.

Terakhir, penulis juga menyadari bahwa tugas akhir ini masih jauh dari kesempurnaan. Oleh karena itu, saran dan kritik sangat penulis harapkan untuk perbaikan tugas akhir ini di kemudian hari.

DAFTAR ISI

ABSTRAK iii

KATA PENGANTAR iv

DAFTAR ISI vi

DAFTAR GAMBAR ix

DAFTAR TABEL x

BAB I PENDAHULUAN I-1

1.1 Latar Belakang I-1

1.2 Rumusan Masalah I-2

1.3 Tujuan I-3

1.4 Batasan Masalah I-3

1.5 Metodologi I-4

1.6 Sistematika Pembahasan I-5

BAB II TINJAUAN PUSTAKA II-1

2.1 Teori Data dan Informasi II-1

2.2 Teori Pemampatan Data II-3

2.3 Metode Pemampatan Data Lempel Ziv Welch (LZW) II-8

2.4 Transformasi Berkas II-14

2.5 Transformasi Burrows Wheeler II-15

2.6 Pengembangan Perangkat Lunak II-19

BAB III DESKRIPSI SOLUSI III-1

3.1 Kompresi Algoritma Lempel Ziv Welch III-1

3.2 Kelemahan Algoritma LZW III-3

3.3 Solusi yang Ditawarkan III-4

3.4 Analisis Solusi III-4

3.5 Faktor-Faktor yang Menentukan III-6
3.6 Analisis Perangkat Lunak III-6
3.6.1 Deskripsi Perangkat Lunak III-6
3.6.2 Spesifikasi Kebutuhan Perangkat Lunak III-7
3.6.3 Diagram *Use Case* III-7
3.6.4 Skenario *Use Case* III-8
3.6.5 Realisasi *Use Case* Tahap Analisis III-11
3.6.6 Diagram Kelas Analisis III-14
3.7 Perancangan Perangkat Lunak III-16
3.7.1 Perancangan Antarmuka III-16
3.7.2 Realisasi *Use Case* Tahap Perancangan III-16
3.7.3 Diagram Kelas Perancangan III-20
3.8 Implementasi Perangkat Lunak III-21
3.8.1 Lingkungan Implementasi III-21
3.8.2 Batasan Implementasi III-22
3.8.3 Implementasi Antarmuka III-22
BAB IV EVALUASI IV-1
4.1 Evaluasi Perangkat Lunak IV-1
4.1.1 Lingkungan Evaluasi IV-1
4.1.2 Tujuan Evaluasi IV-1
4.1.3 Data Uji IV-2
4.1.4 Kasus Uji dan Hasil Evaluasi IV-2
4.2 Evaluasi Algoritma IV-4
4.2.1 Lingkungan Evaluasi IV-4
4.2.2 Tujuan Evaluasi IV-4
4.2.3 Data Evaluasi IV-4

4.2.4 Kasus Uji dan Pengujian IV-7
4.2.5 Analisis Hasil Pengujian IV-12
BAB V KESIMPULAN DAN SARAN V-1
5.1 Kesimpulan V-1
5.2 Saran V-1
DAFTAR REFERENSI x

DAFTAR GAMBAR

Gambar II-1: Representasi Karakter Alfabet di dalam Komputer II-2
Gambar II-2: Klasifikasi Kompresi Data II-4
Gambar II-3: Contoh Kompresi LZW II-12
Gambar II-4: Contoh Dekompresi LZW II-13
Gambar II-5: Klasifikasi Transformasi Berkas II-15
Gambar II-6: Transformasi Burrows Wheeler II-17
Gambar II-7: Detransformasi Burrows Wheeler II-18
Gambar III-1: Diagram *Use Case* III-8
Gambar III-2: *Sequence Diagram* Analisis *Use Case* 01 ..... III-11
Gambar III-3: *Sequence Diagram* Analisis *Use Case* 02 ..... III-12
Gambar III-4: *Sequence Diagram* Analisis *Use Case* 03 ..... III-13
Gambar III-5: *Sequence Diagram* Analisis *Use Case* 04 ..... III-14
Gambar III-6: Diagram Kelas Analisis III-15
Gambar III-7: Antarmuka Utama Perangkat Lunak III-16
Gambar III-8: *Sequence Diagram* Perancangan *Use Case* 01 ..... III-17
Gambar III-9: *Sequence Diagram* Perancangan *Use Case* 02 ..... III-18
Gambar III-10: *Sequence Diagram* Perancangan *Use Case* 03 ..... III-19
Gambar III-11: *Sequence Diagram* Perancangan *Use Case* 04 ..... III-20
Gambar III-12: Diagram Kelas Perancangan III-21
Gambar III-13: *Open File Dialog* III-23
Gambar III-14: *Save File Dialog* III-23
Gambar III-15: *Message Box* ketika tidak memilih mode apapun III-24
Gambar III-16: *Message Box* ketika tidak memilih algoritma satupun III-24
Gambar III-17: *Message Box* ketika tidak memilih berkas III-24
Gambar III-18: *Message Box* konfirmasi pengguna III-24
Gambar III-19: *Message Box* ketika proses eksekusi selesai III-25
Gambar IV-1: Perbandingan Rasio Kompresi Berkas 1-10 ..... IV-10
Gambar IV-2: Perbandingan Rasio Kompresi Berkas 11-20 ..... IV-11
Gambar IV-3: Perbandingan Rasio Kompresi Berkas 21-33 ..... IV-11

DAFTAR TABEL

Tabel II-1: Kompresi Teks dengan Algoritma LZ78 ..... II-10
Tabel II-2: Kompresi Teks dengan Algoritma LZW II-10
Tabel III-1: Kompresi Teks "botbananaboat" III-2
Tabel III-2: Kompresi Teks Hasil Transformasi III-5
Tabel III-3: Daftar Kebutuhan Fungsional III-7
Tabel III-4: Daftar Kebutuhan Non Fungsional III-7
Tabel III-5: Skenario *Use Case* 01 ..... III-8
Tabel III-6: Skenario *Use Case* 02 ..... III-9
Tabel III-7: Skenario *Use Case* 03 ..... III-10
Tabel III-8: Skenario *Use Case* 04 ..... III-10
Tabel IV-1: Data Uji Perangkat Lunak IV-2
Tabel IV-2: Hasil Pengujian Kasus Pertama IV-2
Tabel IV-3: Hasil Pengujian Kasus Kedua IV-3
Tabel IV-4: Data Uji Evaluasi Algoritma IV-5
Tabel IV-5: Hasil Pengujian dengan Algoritma LZW IV-7
Tabel IV-6: Hasil Pengujian dengan Algoritma LZW yang Dioptimisasi IV-8
Tabel IV-7: Hasil Perhitungan Kenaikan Kerja IV-9

BAB I
PENDAHULUAN

Bab ini akan berisi beberapa penjelasan pendahuluan mengenai topik yang diajukan pada tugas akhir ini, antara lain terdiri atas latar belakang penelitian, rumusan masalah, tujuan penelitian, batasan masalah, metodologi penelitian serta sistematika pembahasan tugas akhir.

1.1 Latar Belakang

Sejak dahulu kala, informasi merupakan salah satu kebutuhan manusia yang paling mendasar. Salah satu penyebab mengapa informasi ini sangatlah penting karena manusia memiliki kodrat sebagai makhluk sosial. Pada dasarnya manusia memerlukan informasi untuk menentukan tindakan apa yang harus dilakukan. Informasi juga merupakan hal yang vital di dalam proses komunikasi.

Seiring dengan perkembangan teknologi, pertumbuhan jumlah data maupun informasi berkembang lebih pesat dibandingkan dengan kapasitas media penyimpanan. Karena keterbatasan kapasitas penyimpanan tersebut, maka muncul gagasan untuk menyimpan data atau informasi tersebut dengan lebih efisien(Witten, 1999). Dengan menyimpan data secara lebih efisien, maka manusia dapat menghemat ruang penyimpanan sehingga jumlah data yang disimpan bisa lebih banyak. Salah satu solusi untuk menyimpan data secara lebih efisien ialah dengan menggunakan teknik kompresi data.

Ide dasar dari pemampatan data ialah banyaknya redundansi (pengulangan) pada representasi data. Pada dasarnya, kompresi data atau pemampatan data ialah memampatkan ukuran data sehingga menjadi lebih kecil dibandingkan sebelumnya. Hal ini bisa dilakukan karena ukuran data akan di-*encode* menjadi bit-bit yang lebih kecil dengan menggunakan metode tertentu.

Ada banyak algoritma pemampatan data, diantaranya yang terkenal antara lain: Huffman Coding, LZ77 (*Sliding Window)*, LZ78, dan LZW. Pada tugas akhir ini

digunakan algoritma LZW karena algoritma ini memiliki beberapa kelebihan antara lain merupakan perbaikan dari algoritma LZ77 dan LZ78, mampu mengkompresi data dengan cepat, serta tidak rumit untuk diimplementasikan. Namun, algoritma ini memiliki kelemahan salah satunya ialah sulitnya memanajemen tabel string. Masalah inilah yang menyebabkan algoritma LZW harus mengalami proses optimisasi agar memberikan hasil yang lebih baik.

Untuk mengoptimisasi algoritma pemampatan data, salah satu metode yang digunakan adalah metode transformasi Burrows Wheeler(Simpson, 2008). Optimisasi algoritma pemampatan data bertujuan untuk menghasilkan data yang dimampatkan secara lebih cepat maupun dengan ukuran yang lebih kecil dibandingkan dengan kinerja algoritma sebelum dioptimisasi. Metode transformasi Burrows Wheeler mentransformasikan isi dari berkas teks berdasarkan permutasi seluruh kemungkinan rotasi dari karakter-karakter di dalam berkas teks. Hasilnya ialah berkas teks yang akan dimampatkan mengalami perubahan bentuk, dimana bagian-bagian yang memiliki kemiripan akan ditempatkan secara berdekatan. Pada tugas akhir ini digunakan optimisasi dengan metode transformasi Burrows Wheeler karena metode ini tidak membutuhkan kamus tambahan sehingga tidak dibutuhkan waktu yang lama dalam proses implementasinya.

Tujuan dari tugas akhir ini yaitu membandingkan bagaimana kinerja algoritma LZW pra maupun pasca proses optimisasi. Algoritma tersebut akan mengalami optimisasi dengan metode transformasi Burrows Wheeler, sehingga hasil pemampatan datanya secara teoritis akan menjadi lebih baik. Untuk proses perbandingan hasil optimisasinya, algoritma LZW akan diimplementasi ke dalam suatu perangkat lunak sederhana, sehingga memudahkan proses pengujian dan evaluasi terhadap kinerja algoritma sebelum maupun sesudah dioptimisasi.

1.2 Rumusan Masalah

Ide yang mendasari pengerjaan tugas akhir ini ialah untuk melakukan optimisasi terhadap algoritma kompresi data yang cukup popular, yaitu algoritma Lempel Ziv

Welch (LZW). Optimisasi algoritma diharapkan dapat menghasilkan data hasil kompresi yang lebih termampatkan.

Secara rinci, tahapan dari rumusan masalah di atas ialah:

1. Bagaimana melakukan optimisasi terhadap kinerja algoritma LZW dengan menggunakan metode transformasi Burrows Wheeler.
2. Bagaimana melakukan implementasi algoritma kompresi dan dekompresi LZW serta transformasi dan detransformasi Burrows Wheeler pada perangkat lunak.
3. Bagaimana kinerja algoritma LZW sebelum maupun setelah mengalami optimisasi.

1.3 Tujuan

Tujuan yang ingin dicapai dari pengerjaan tugas akhir ini ialah membandingkan kinerja algoritma LZW sebelum dan sesudah dioptimisasi sehingga dapat diketahui manakah hasil yang lebih baik sebelum atau sesudah dioptimisasi.

Adapun tahapan untuk mencapai tujuan tersebut ialah:

1. Mempelajari dan memahami algoritma LZW dan transformasi Burrows Wheeler.
2. Melakukan optimisasi terhadap kinerja algoritma LZW dengan menggunakan transformasi Burrows Wheeler.
3. Melakukan implementasi algoritma LZW dan transformasi Burrows Wheeler untuk melakukan pengujian kinerja algoritma.
4. Membandingkan hasil pengujian kinerja algoritma sebelum maupun sesudah dioptimisasi dengan transformasi Burrows Wheeler.

1.4 Batasan Masalah

Batasan masalah dalam studi dan penyusunan tugas akhir ini adalah:

1. Kompresi maupun dekompresi data terbatas pada berkas berjenis *plain text*.
2. Pengukuran kinerja dilakukan hanya dari sisi rasio kompresi.

1.5 Metodologi

Dalam mengerjakan tugas akhir ini, ada beberapa langkah yang dilakukan, yaitu:

1. Studi Literatur

Pada tahap ini dilakukan pembelajaran mengenai teori dasar yang digunakan pada tugas akhir ini, antara lain tentang teori kompresi data, algoritma LZW, dan transformasi Burrows Wheeler. Literatur yang digunakan antara lain bahan dari situs internet, makalah ilmiah, buku teks, maupun tugas akhir yang relevan dengan topik kompresi data.

2. Analisis

Pada tahap ini dilakukan analisis mengenai optimisasi yang dilakukan, antara lain hipotesis optimisasi dengan menggunakan transformasi Burrows Wheeler dapat meningkatkan kinerja algoritma kompresi LZW. Pada tahap ini juga dilakukan analisis terhadap perangkat lunak yang dibangun untuk melakukan pengujian terhadap kinerja kedua algoritma sebelum dan sesudah dioptimisasi.

3. Perancangan dan Implementasi

Pada tahap ini dilakukan perancangan serta implementasi algoritma kompresi dalam bentuk perangkat lunak. Perangkat lunak yang dibangun berdasarkan hasil analisis yang telah dilakukan sebelumnya. Perangkat lunak ini berguna untuk membantu mengukur kinerja algoritma sebelum maupun sesudah dioptimisasi dengan transformasi Burrows Wheeler.

4. Pengujian dan Evaluasi

Pada tahap ini dilakukan pengujian perangkat lunak yang telah dibuat untuk menguji apakah perangkat lunak bisa berfungsi sesuai rancangan yang telah dibuat. Setelah itu akan dilakukan evaluasi kinerja algoritma LZW sebelum maupun sesudah dioptimisasi.

5. Kesimpulan dan Saran

Pada tahap ini dilakukan penarikan kesimpulan studi yang telah dilakukan. Akan ditarik kesimpulan apakah hipotesis awal penelitian terbukti ataupun tidak terbukti. Kemudian akan diberikan saran bagi penelitian selanjutnya yang memiliki relevansi dengan tugas akhir ini.

1.6 Sistematika Pembahasan

Laporan tugas akhir ini disusun dengan sistematika pembahasan sebagai berikut:

BAB I Merupakan bab pendahuluan yang tersusun atas latar belakang tugas akhir, rumusan masalah, tujuan tugas akhir, batasan masalah, metodologi pengerjaan tugas akhir, dan sistematika penulisan tugas akhir.

BAB II Merupakan bab tinjauan pustaka yang berisi teori-teori yang diambil dari berbagai literatur dan merupakan dasar pengerjaan tugas akhir ini. Teori-teori yang dibahas antara lain teori data dan informasi, teori pemampatan data, algoritma LZW, serta transformasi Burrows Wheeler.

BAB III Merupakan bab hasil analisis mengenai deskripsi solusi yang diajukan, yaitu mengenai optimisasi yang akan dilakukan pada algoritma LZW. Selain itu, akan terdapat analisis dan perancangan perangkat lunak yang akan dibangun.

BAB IV Merupakan bab evaluasi yang berisikan evaluasi terhadap perangkat lunak yang telah dibangun. Evaluasi yang dilakukan antara lain evaluasi terhadap kebutuhan fungsional perangkat lunak, evaluasi terhadap kinerja algoritma, dan data uji yang digunakan.

BAB V Merupakan bab penutup yang berisikan kesimpulan tugas akhir serta saran untuk pengembangan di kemudian hari.

BAB II
TINJAUAN PUSTAKA

Bab ini akan berisi pembahasan mengenai teori-teori hasil studi literatur yang melandasi pembuatan tugas akhir ini. Hal-hal yang akan dibahas diantaranya ialah teori data dan informasi, teori pemampatan (kompresi) data, algoritma Lempel-Ziv-Welch, transformasi berkas, serta transformasi Burrows wheeler.

2.1 Teori Data dan Informasi

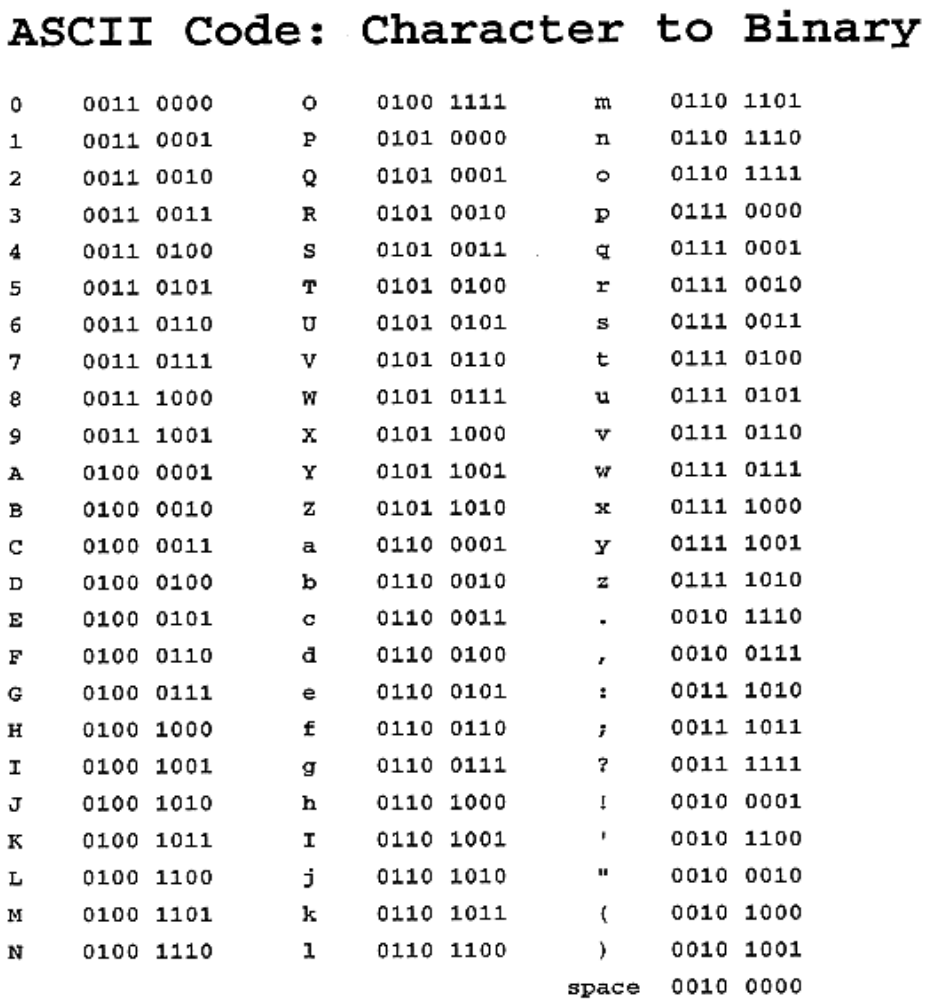
Kata “data” berasal dari bahasa Yunani “*datum*” yang berarti fakta, dan di dalam kamus bahasa Inggris ditulis dengan “*data*”. “Data” yang digunakan dalam bahasa Indonesia berasal dari bahasa Inggris tersebut, tetapi harus diingat, “*data*” dalam bahasa Inggris sudah bersifat majemuk, karena tidak ada kata “*datas*” dalam bahasa Inggris. Sehingga tidaklah tepat bila kita menuliskan kata data yang dimajemukkan, seperti data-data, kumpulan data, dan sejenisnya.

Pada dasarnya data dapat diartikan sebagai representasi dari fakta. Awalnya, data hanyalah merupakan kumpulan fakta yang belum memiliki makna. Untuk memperoleh makna dari suatu data, data tersebut harus mengalami pemrosesan, sehingga akan berubah menjadi informasi(Witten, 2009).

Teori informasi yang dikembangkan oleh Claude E. Shannon mempelajari aspek kuantitatif dari informasi(Hankerson, 2003). Dengan menggunakan teori ini, dapat dihitung banyaknya informasi yang terdapat dalam suatu data dengan angka yang tepat. Karena itulah didefinisikan unit dari informasi yang disebut dengan bit. Saat ini sudah menjadi hal yang lumrah untuk menyebutkan ukuran dari suatu informasi dalam satuan bit. Teori informasi banyak diaplikasikan di berbagai bidang, diantaranya pemrosesan bahasa alami, kriptografi, komputasi kuantum, inferensi statistik, dan lain-lain.

Dalam ilmu komputer, data digunakan sebagai informasi yang dibutuhkan oleh komputer agar dapat menjalankan fungsinya. Di dalam komputer, data disimpan

dalam format *binary* dan direpresentasikan oleh kumpulan angka 0 dan 1. Setiap angka 0 maupun 1 menghabiskan ruang sebesar 1 bit di dalam media penyimpanan. Karakter-karakter dalam bentuk *binary* pada komputer dapat dilihat pada gambar II-1.



ASCII Code: Character to Binary

0	0011 0000	O	0100 1111	m	0110 1101
1	0011 0001	P	0101 0000	n	0110 1110
2	0011 0010	Q	0101 0001	o	0110 1111
3	0011 0011	R	0101 0010	p	0111 0000
4	0011 0100	S	0101 0011	q	0111 0001
5	0011 0101	T	0101 0100	r	0111 0010
6	0011 0110	U	0101 0101	s	0111 0011
7	0011 0111	V	0101 0110	t	0111 0100
8	0011 1000	W	0101 0111	u	0111 0101
9	0011 1001	X	0101 1000	v	0111 0110
A	0100 0001	Y	0101 1001	w	0111 0111
B	0100 0010	Z	0101 1010	x	0111 1000
C	0100 0011	a	0110 0001	y	0111 1001
D	0100 0100	b	0110 0010	z	0111 1010
E	0100 0101	c	0110 0011	.	0010 1110
F	0100 0110	d	0110 0100	,	0010 0111
G	0100 0111	e	0110 0101	:	0011 1010
H	0100 1000	f	0110 0110	;	0011 1011
I	0100 1001	g	0110 0111	?	0011 1111
J	0100 1010	h	0110 1000	!	0010 0001
K	0100 1011	I	0110 1001	'	0010 1100
L	0100 1100	j	0110 1010	"	0010 0010
M	0100 1101	k	0110 1011	(	0010 1000
N	0100 1110	l	0110 1100	)	0010 1001
				space	0010 0000

Gambar II-1: Representasi Karakter Alfabet di dalam Komputer

Jumlah data yang terus bertambah tidak diimbangi dengan peningkatan kapasitas media penyimpanan data. Oleh karena itu, manusia mencari cara agar dapat menyimpan data secara lebih efektif untuk memanfaatkan kapasitas media penyimpanan secara optimal. Salah satu gagasan yang muncul ialah bagaimana merepresentasikan data dalam bentuk yang lebih ringkas, sehingga data dapat disimpan dalam ukuran yang lebih kecil(Hankerson, 2003).

2.2 Teori Pemampatan Data

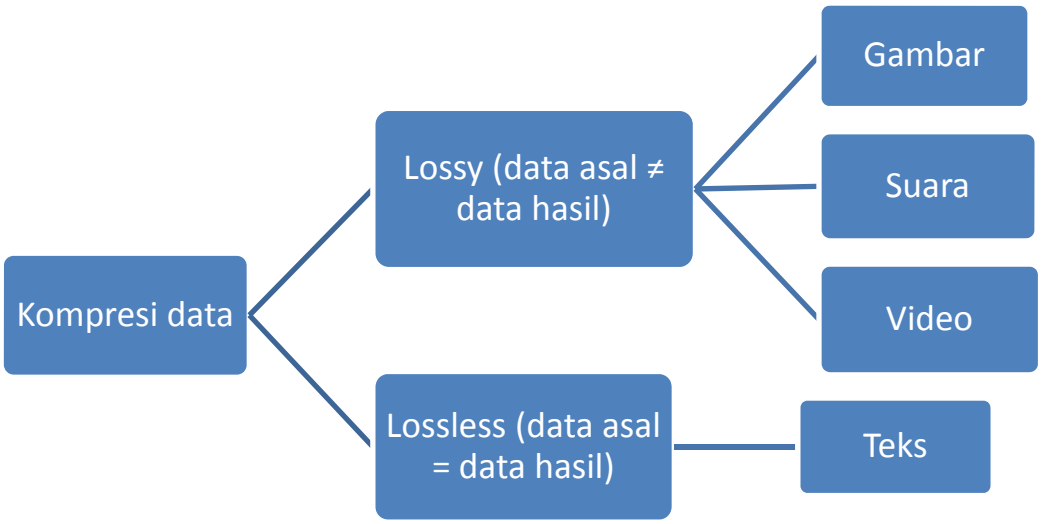
Pemampatan data dapat diartikan sebagai proses mengubah aliran data masukan ke dalam aliran data lain yang lebih kecil. Pemampatan data, atau disebut juga kompresi data memiliki berbagai matode berdasarkan ide yang berbeda-beda. Proses pemampatan data menjadi populer dalam perkembangan ilmu komputer dikarenakan oleh hal-hal berikut(Salomon, 2007):

1. Manusia memiliki kecenderungan untuk mengumpulkan data secara terus-menerus tanpa menghapus data sebelumnya. Hal ini tentunya menjadi masalah karena kapasitas media penyimpanan semakin lama semakin berkurang. Akibatnya, data baru yang akan disimpan tidak lagi memiliki tempat penyimpanan. Dengan menggunakan metode pemampatan data, jumlah data yang disimpan dalam media penyimpanan dapat ditingkatkan.
2. Waktu transfer data dapat dipercepat apabila data berukuran lebih kecil. Hal ini sangat bermanfaat, terutama karena pada era internet sekarang, kecepatan transfer data menjadi *constraint* yang sangat penting untuk para pengguna internet. Dengan menggunakan metode pemampatan data yang sesuai, data yang akan ditransfer dapat dikurangi ukurannya dan menyebabkan waktu transfer data menjadi berkurang.

Prinsip dasar metode pemampatan data ialah dengan menyingkirkan bagian yang dianggap redundan dari aliran data masukan. Hal ini berdasarkan prinsip bahwa data memiliki struktur tertentu, dimana struktur ini dapat dimanipulasi untuk mendapatkan representasi data dalam ukuran yang lebih kecil. Karena struktur data dapat berubah antara berbagai jenis data, maka tidak ada algoritma pemampatan data yang bekerja secara optimal untuk seluruh jenis data.

Metode pemampatan data dapat dibagi menjadi dua bagian, yaitu pemampatan data *lossless* dan pemampatan data *lossy*(Salomon, 2007). Pada pemampatan data *lossy*, data yang dimampatkan akan memiliki perbedaan dengan data asal karena pada metode ini sebagian informasi pada data asal akan dihilangkan. Metode pemampatan data *lossy* banyak digunakan untuk memampatkan data video, suara, dan gambar. Hal ini dikarenakan pada tipe data tersebut, tidak akan terlihat perbedaan yang signifikan apabila sebagian informasi dari tipe data tersebut dihilangkan. Namun lain halnya

pada data teks, dimana perubahan informasi, bahkan hanya sebesar 1 bit, dapat mengakibatkan perbedaan yang signifikan antara data awal dan data hasil pemampatan. Karena itulah, pada pemampatan data teks sebaiknya menggunakan metode *lossless*, yaitu data asal sama dengan data hasil pemampatan. Untuk memahami klasifikasi kompresi data, perhatikan gambar II-2.



Gambar II-2: Klasifikasi Kompresi Data

Beberapa hal yang dapat digunakan untuk mengukur kinerja dari sebuah algoritma pemampatan data ialah sebagai berikut(Salomon, 2007):

1. Rasio pemampatan data (*compression ratio*)

Adalah ukuran hasil pemampatan data dibagi dengan ukuran berkas semula. Semakin kecil nilai rasio pemampatan data, semakin efektif kinerja algoritma kompresi tersebut. Sebagai contoh, apabila nilai rasio 0,7 berarti algoritma menghasilkan data keluaran berukuran 70% dari ukuran semula.

$$R_0 = \frac{\text{Ukuran keluaran aliran data}}{\text{Ukuran masukan aliran data}} \qquad \text{II.1}$$

2. Faktor pemampatan data (*compression factor*)

Adalah inverse dari rasio pemampatan data. Apabila nilainya lebih kecil dari satu, maka algoritma pemampatan data tersebut tidak layak untuk digunakan. Karena

apabila nilai kecil dari satu maka terjadi ekspansi data, dan nilai lebih besar dari satu maka terjadi pemampatan data.

$$F_o = \frac{\text{Ukuran masukan aliran data}}{\text{Ukuran keluaran aliran data}}$$

II.2

3. Kecepatan kompresi (*compression speed*)

Adalah perbandingan ukuran berkas yang akan dimampatkan dengan waktu yang dibutuhkan untuk memampatkan berkas berkas tersebut. Apabila kecepatan kompresi kecil maka waktu yang dibutuhkan untuk memampatkan data menjadi lama sehingga algoritma pemampatan data tersebut tidak banyak digunakan.

$$Kecepatan\ kompresi = \frac{\text{Ukuran berkas semula}}{\text{Waktu kompresi}}$$

II.3

4. Kecepatan dekompresi (*decompression speed*)

Adalah perbandingan ukuran berkas hasil dimampatkan dengan waktu yang dibutuhkan untuk mendekompresi berkas berkas tersebut menjadi bentuk berkas awal. Algoritma pemampatan data yang baik haruslah memiliki kecepatan kompresi dan dekompresi yang saling berimbang.

$$Kecepatan\ dekompresi = \frac{\text{Ukuran berkas hasil pemampatan}}{\text{Waktu dekompresi}}$$

II.4

5. Entropi(Shannon, 1948)

Menghitung entropi erat kaitannya dengan teori entropi informasi dari Shannon. Nilai entropi merepresentasikan jumlah minimum bit yang diperlukan untuk meng-*encode*berkas. Persamaan yang digunakan untuk menghitung entropi ialah:

$$H(X) = -\sum_{i=0}^{N-1} p_i \log_2 p_i$$

II.5

Dimana i menyatakan karakter ke-i dan N menyatakan jumlah jenis karakter. Makin kecil nilai H(X) maka nilai entropi semakin kecil, artinya algoritma kompresi semakin baik.

Secara umum prinsip dasar dari pemampatan data ialah mengkodekan karakter-karakter yang memiliki redundansi (pengulangan) di dalam berkas. Setelah itu pemampatan data mulai memanfaatkan redundansi pada representasi karakter ASCII. Representasi karakter dengan ASCII memiliki redundansi yang besar, karena ASCII mengkodekan setiap karakter dengan panjang yang tetap. Dengan memanfaatkan redundansi tersebut, muncul gagasan untuk melakukan pengkodean karakter dengan panjang bit variatif, yang dikenal dengan sebutan *varibale size code*.

Untuk mendapatkan kinerja yang baik dari pengkodean bit dengan panjang variatif, maka diperlukan pengetahuan tentang jumlah kemunculan setiap karakter dalam suatu berkas berkas, agar setiap karakter dapat direpresentasikan dengan panjang kode yang berbeda. Karakter yang memiliki tingkat kemunculan tinggi direpresentasikan dengan kode yang lebih pendek, sedangkan karakter yang memiliki tingkat kemunculan rendah direpresentasikan dengan kode yang lebih panjang. Itulah mengapa pendekatan statistik digunakan, yaitu untuk menghitung kemunculan setiap karakter pada berkas. Salah satu pendekatan statistik yang populer ialah metode pemampatan data Huffman.

Karena pendekatan secara statistik sangat bergantung kepada permodelan statistik yang dibuat, maka manusia menciptakan pendekatan lain, yaitu pendekatan dengan menggunakan kamus. Pendekatan dengan menggunakan kamus tidak bergantung pada model statistik dan tidak menggunakan kode dengan panjang variatif (*variable size code*). Pada pendekatan ini yang dilakukan ialah membangun sebuah kamus berdasarkan berkas masukan, dan kamus ini berguna untuk melakukan encoding string masukan menjadi token. Kamus ini dapat bersifat statis, yaitu kamus mengizinkan penambahan simbol saja, maupun bersifat adaptif atau disebut juga dinamis, yaitu kamus mengizinkan penambahan simbol maupun pengurangan simbol dari dalam kamus.

Langkah-langkah umum pada pemampatan data dengan menggunakan pendekatan kamus ialah:

1. Melakukan pembacaan teks
2. Membuat kamus berdasarkan kemunculan kata pada berkas

3. Mengkodekan setiap kata berdasarkan indeks lokasi kata pada kamus

Sebelum mengambil tugas akhir dengan topik ini, telah ditelaah beberapa literatur yang memiliki keterkaitan dengan penelitian di bidang algoritma pemampatan data. Beberapa literatur yang terkait adalah:

1. Tugas akhir Arinta Praimandini Auza, berjudul "Analisis Perbandingan Algoritma LZ78 dan *Arithmetic Coding* pada Kompresi File Teks"

Pada tugas akhir ini saudari Arinta membandingkan kinerja algoritma kompresi LZ78 dan algoritma kompresi *Arithmetic Coding*. Hasil yang diperoleh ialah algoritma *Arithmetic Coding* mengkompresi berkas menjadi lebih kecil, tetapi algoritma LZ78 lebih unggul dari segi waktu kompresi. Selain itu saudari Arinta menyimpulkan bahwa algoritma LZ78 tidak cocok untuk mengkompresi berkas berukuran besar, karena semakin besar ukuran berkas algoritma LZ78 semakin tidak akurat. Perbedaan mendasar dari LZ77 dan LZ78 ialah pada proses dekompresi algoritma LZ78, memungkinkan pembacaan karakter yang dimulai secara acak pada berkas, tidak harus dari awal berkas.

2. Tugas akhir Adi Purwanto Sujarwadi, berjudul "Optimasi Kinerja Algoritma Pemampatan Data Lempel-Ziv dengan Metode Transformasi Burrows Wheeler"

Pada tugas akhir ini, saudara Adi melakukan optimisasi pada algoritma LZ77 (*sliding window)* dengan menggunakan transformasi Burrows Wheeler. Hasil yang diperoleh ialah, preproses teks dengan transformasi dapat meningkatkan kinerja algoritma kompresi LZ77, yaitu menghasilkan file hasil kompresi yang berukuran lebih kecil untuk berkas teks biasa. Namun, kelemahan dari algoritma LZ77 yang dioptimisasi ialah dapat menurunkan kinerja dari kompresi untuk berkas berjenis *calgary corpus*.

Oleh karena itu, tugas akhir yang diambil secara ringkas melanjutkan tugas akhir saudara Adi. Karena tugas akhir saudara Adi menggunakan algoritma LZ77, maka tugas akhir ini menggunakan algoritma LZW yang merupakan penyempurnaan dari algoritma LZ77. Selain itu juga, dilakukan pengujian optimisasi algoritma pada berkas berjenis *Calgary Corpus*.

2.3 Metode Pemampatan Data Lempel Ziv Welch (LZW)

Algoritma pemampatan data Lempel Ziv Welch adalah algoritma pemampatan data dengan menggunakan pendekatan kamus(Ziv, 1997). Kamus ini digunakan untuk menyimpan string yang telah ditemukan sebelumnya. Algoritma yang ditemukan oleh Abraham Lempel, Jacob Ziv, dan Terry Welch ini dipublikasikan sebagai implementasi yang lebih baik dari algoritma LZ78.

Untuk memahami algoritma LZW secara lebih mendalam, pertama-tama akan dibahas perbandingan algoritma LZW dengan algoritma LZ78. Pada awalnya kamus pada algoritma LZ78 kosong dan ukurannya terbatas oleh memori yang dapat digunakan. Pemampatan data dengan menggunakan algoritma LZ78 ialah sebagai berikut(Salomon, 2007):

1. Kamus diawali dengan karakter null yang memiliki indeks 0
2. Baca karakter berikutnya pada *file input*. Setiap string yang ditambahkan pada kamus akan diberi indeks 1, 2, dan seterusnya.
3. Saat karakter x dibaca dari *file input*, kamus mencari sebuah token yang berisi string x. Token bisa ditulis dalam bentuk (X,Y), dimana X berupa bilangan bulat yang menunjukkan penunjuk pada kamus, dan Y berupa string.
   a) Jika string tidak ditemukan, maka token (0,x) ditambahkan di dalam kamus.
   b) Jika string x ditemukan, misalnya pada posisi k, maka string berikutnya y dibaca, dan kamus mencari token yang berisi xy. Jika tidak ditemukan, maka token (k,y) ditambahkan di dalam kamus. Sedangkan apabila karakter ditemukan, maka proses pembacaan berlanjut terus hingga ditemukan karakter yang tidak berada di dalam kamus.
4. Kemudian ulangi dari langkah kedua hingga seluruh isi berkas terbaca.

Sedangkan pada algoritma pemampatan data LZW, kamus tidak diinisialisasi dengan karakter null, melainkan telah diisi dengan 0-255 karakter ASCII atau 256 8-bit byte. Perbedaan yang lain ialah, token pada LZW hanya terdiri atas satu elemen, yaitu penunjuk pada kamus, karena kamus pada LZW sudah diinisialisasi sehingga karakter yang akan dibaca berikutnya pasti telah ada di dalam kamus. Prinsip utama algoritma ini ialah memampatkan karakter satu per satu dan mengakumulasikannya ke dalam

string I Pemampatan data dengan algoritma LZW adalah sebagai berikut(Salomon, 2007):

1. Inisialisasi kamus dengan 256 entri di dalam kamus
2. Baca karakter berikutnya pada berkas input.
3. Setiap karakter dikonkatenasi dengan string I.

a) Apabila I terus menerus ditemukan di dalam kamus, maka proses pembacaan terus berlanjut.

b) Apabila konkatenasi karakter dengan string I tidak ditemukan di dalam kamus, misalnya Ix, maka kamus menambahkan token baru di dalam kamus, yaitu penunjuk ke string I, menyimpan string Ix di dalam kamus, dan menginisialisasi string I ke karakter x.

4. Ulangi mulai langkah yang kedua hingga pembacaan berkas input selesai.

Untuk lebih jelasnya, *pseudo-code* kompresi LZW adalah sebagai berikut.

```
STRING = get input character
WHILE there are still input characters DO
    CHARACTER = get input character
    IF STRING+CHARACTER is in the string table then
        STRING = STRING+character
    ELSE
        output the code for STRING
        add STRING+CHARACTER to the string table
        STRING = CHARACTER
    END of IF
END of WHILE
output the code for STRING
```

Sedangkan *pseudo-code* dekompresi LZW adalah sebagai berikut:

```
Read OLD_CODE
output OLD_CODE
WHILE there are still input characters DO
    Read NEW_CODE
    STRING = get translation of NEW_CODE
    output STRING
    CHARACTER = first character in STRING
    add OLD_CODE + CHARACTER to the translation table
    OLD_CODE = NEW_CODE
END of WHILE
```

Untuk lebih memahami perbedaan algoritma LZ78 dan LZW, diberikan contoh pemampatan data pada string: “sir sid eastman easily”

Pemampatan data dengan algoritma LZ78 adalah sebagai berikut:

Tabel II-1: Kompresi Teks dengan Algoritma LZ78

Kamus		Token
0	null	
1	“s”	(0,”s”)
2	“i”	(0,”i”)
3	“r”	(0,”r”)
4	“ “	(0,” “)
5	“si”	(1,”i”)
6	“d”	(0,”d”)
7	“ e”	(4,” e”)
8	“a”	(0,”a”)
9	“st”	(1,”t”)
10	“m”	(0,”m”)
11	“an”	(8,”n”)
12	“ ea”	(7,”a”)
13	“sil”	(5,”1”)
14	“y”	(0,”y”)

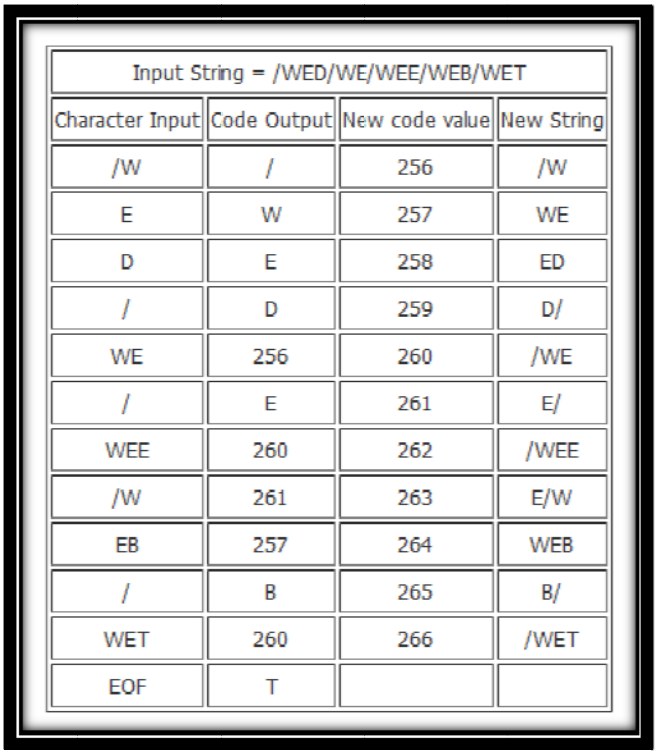
Pemampatan dengan algoritma LZW adalah sebagai berikut:

Tabel II-2: Kompresi Teks dengan Algoritma LZW

I	Ada di dalam kamus?	Entri baru	Keluaran
s	Y		
si	T	256-si	115(s)
i	Y		
ir	T	257-ir	105(i)
r	Y		
r” “	T	258-r” “	114(r)
“ “	Y		
“ “s	T	259-“ “s	32(“ “)
s	Y		

I	Ada di dalam kamus?	Entri baru	Keluaran
si	Y		
sid	T	260-sid	256(si)
d	Y		
d” “	T	261-d” “	100(d)
“ “	Y		
“ “e	T	262-“ “e	32(“ “)
e	Y		
ea	T	263-ea	101(e)
a	Y		
as	T	264-as	97(a)
s	Y		
st	T	265-st	115(s)
t	Y		
tm	T	266-tm	116(t)
m	Y		
ma	T	267-ma	109(m)
a	Y		
an	T	268-an	97(a)
n	Y		
n” “	T	269-n” “	110(n)
“ “	Y		
“ “e	Y		
“ “ea	T	270-“ “ea	262(“ “e)
a	Y		
as	Y		
asi	T	271-asi	264(as)
i	Y		
il	T	272-il	105(i)
l	Y		
ly	T	273-ly	108(l)
y	Y		121(y)

Untuk memahami perbedaan antara kompresi dan dekompresi LZW, perhatikan gambar II-3 dan II-4 berikut:



Input String = /WED/WE/WEE/WEB/WET			
Character Input	Code Output	New code value	New String
/W	/	256	/W
E	W	257	WE
D	E	258	ED
/	D	259	D/
WE	256	260	/WE
/	E	261	E/
WEE	260	262	/WEE
/W	261	263	E/W
EB	257	264	WEB
/	B	265	B/
WET	260	266	/WET
EOF	T		

Gambar II-3: Contoh Kompresi LZW

String "/WED/WE/WEE/WEB/WET" akan dikompresi menjadi "/W E D 256 E 260 261 257 B 260 T". Spasi disini dituliskan sebagai pemisah antara kode pada *string* hasil kompresi. Untuk mengembalikan*string* hasil kompresi menjadi *string* awal, proses dekompresinya adalah sebagai berikut:

Decoder akan membaca setiap kode satu per satu. Setiap kode akan ditranslasikan sesuai kamus. Untuk setiap pembacaan kode, *decoder* akan menambahkan kode sebelumnya + karakter pertama dari *string* hasil translasi ke dalam kamus. Untuk lebih jelasnya perhatikan gambar II-4. Translasi kode akan mengacu pada kamus yang telah dibuat. Contohnya, kode 256 akan ditranslasi sebagai "/W".

Input Codes: / W E D 256 E 260 261 257 B 260 T				
Input/ NEW_CODE	OLD_CODE	STRING/ Output	CHARACTER	New table entry
/	/	/		
W	/	W	W	256 = /W
E	W	E	E	257 = WE
D	E	D	D	258 = ED
256	D	/W	/	259 = D/
E	256	E	E	260 = /WE
260	E	/WE	/	261 = E/
261	260	E/	E	262 = /WEE
257	261	WE	W	263 = E/W
B	257	B	B	264 = WEB
260	B	/WE	/	265 = B/
T	260	T	T	266 = /WET

Gambar II-4: Contoh Dekompresi LZW

LZW dipilih sebagai algoritma kompresi dalam tugas akhir ini karena beberapa alasan, antara lain:

1. LZW sangat cocok untuk digunakan pada kompresi berkas berkas teks, karena sifatnya yang *lossless*.
2. LZW memiliki waktu kompresi yang cepat dan mudah untuk diimplementasikan.
3. Kamus pada LZW sudah diinisialisasi sehingga menjadikan proses kompresi lebih cepat.

Namun algoritma LZW memiliki kelemahan diantaranya:

1. Pada berkas berkas yang karakter-karakternya tersebar acak, algoritma LZW justru dapat menyebabkan ukuran berkas menjadi lebih besar.
2. Manajemen tabel string, yaitu kamus akan menjadi sulit apabila karakter-karakter di dalam berkas tersebar secara acak.

Kelemahan-kelemahan itulah yang menyebabkan algoritma LZW akan dioptimisasi dengan transformasi Burrows Wheeler.

2.4 Transformasi Berkas

Transformasi berkas adalah proses melakukan modifikasi pada berkas sehingga mengubah berkas menjadi bentuk yang lebih mudah untuk dimampatkan. Transformasi berkas biasanya dilakukan sebelum memampatkan data agar bisa mendapatkan hasil pemampatan data yang lebih baik.

Ada 3 jenis transformasi yang seringkali digunakan dalam memproses berkas sebelum dimampatkan, yaitu(Radescu, 2009):

1. Transformasi Burrows Wheeler

Transformasi Burrows Wheeler menggunakan matriks untuk menyusun permutasi kemunculan karakter dalam berkas, sehingga akhirnya pola repetisi yang tadinya terletak secara tersebar dalam berkas teks dapat dikumpulkan pada lokasi yang berdekatan.

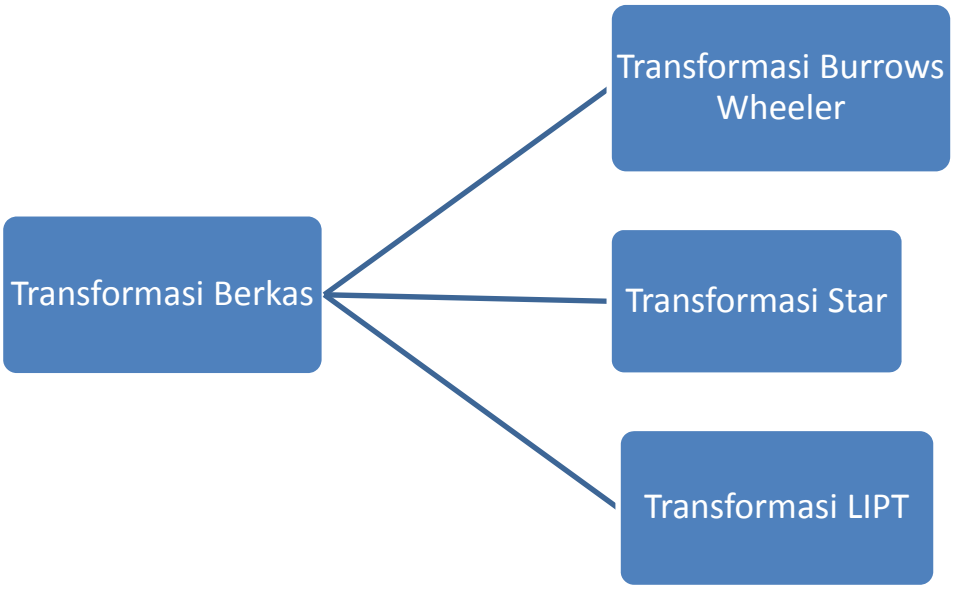
2. Transformasi Star

Transformasi Star berdasarkan pada pendefinisian tanda yang unik unuk setiap karakter dan menggantikan karakter tersebut dengan karakter khusus, contohnya karakter *. Transformasi ini menggunakan kamus untuk melakukan transformasi pada berkas.

3. Transformasi LIPT (Length Index Preserving Transform)

Transformasi LIPT merupakan pengembangan dari transformasi Star. Transformasi LIPT menggunakan kamus dan memiliki kecepatan yang lebih baik dibandingkan transformasi Star. Namun sayangnya transformasi ini belum popular digunakan.

Untuk lebih jelasnya, perhatikan gambar II-5 tentang klasifikasi transformasi berkas.



Gambar II-5: Klasifikasi Transformasi Berkas

2.5 Transformasi Burrows Wheeler

Transformasi Burrows Wheeler adalah salah satu metode yang digunakan untuk melakukan transformasi pada berkas teks. Transformasi yang terjadi ialah menjadikan bagian-bagian yang banyak berulang pada teks menjadi berdekatan. Pada proses transformasi, berkas teks hasil transformasi akan menjadi lebih besar karena ditambahkan variabel lokasi baris pada hasil transformasinya. Bagian yang saling berulang akan diletakkan berdekatan sehingga menjadikan berkas teks tersebut lebih mudah untuk dimampatkan(Simpson, 2008).

Cara kerja algoritma transformas Burrows Wheeler adalah sebagai berikut:

1. Membaca string x
2. Membuat tabel yang berisi semua kemungkinan rotasi dari string x
3. Mengurutkan kolom tabel secara alfabetis
4. Mengembalikan kolom terakhir dan nomor baris yang mengandung berkas awal sebagai berkas yang telah ditransformasi

Cara untuk mengembalikan berkas hasil transformasi adalah sebagai berikut:

1. Membuat sebuah tabel kosong
2. Membaca string yang ditransformasi y
3. Ulangi sebanyak panjang y:

a) Masukkan string y sebagai kolom sebelum kolom pertama
b) Lakukan pengurutan secara alfabetis
4. Jadikan baris yang bersesuaian dengan nomor baris yang terdapat pada berkas yang telah ditransformasi sebagai keluaran

Transformasi Burrows Wheeler memiliki beberapa keunggulan sehingga digunakan pada tugas akhir ini antara lain:

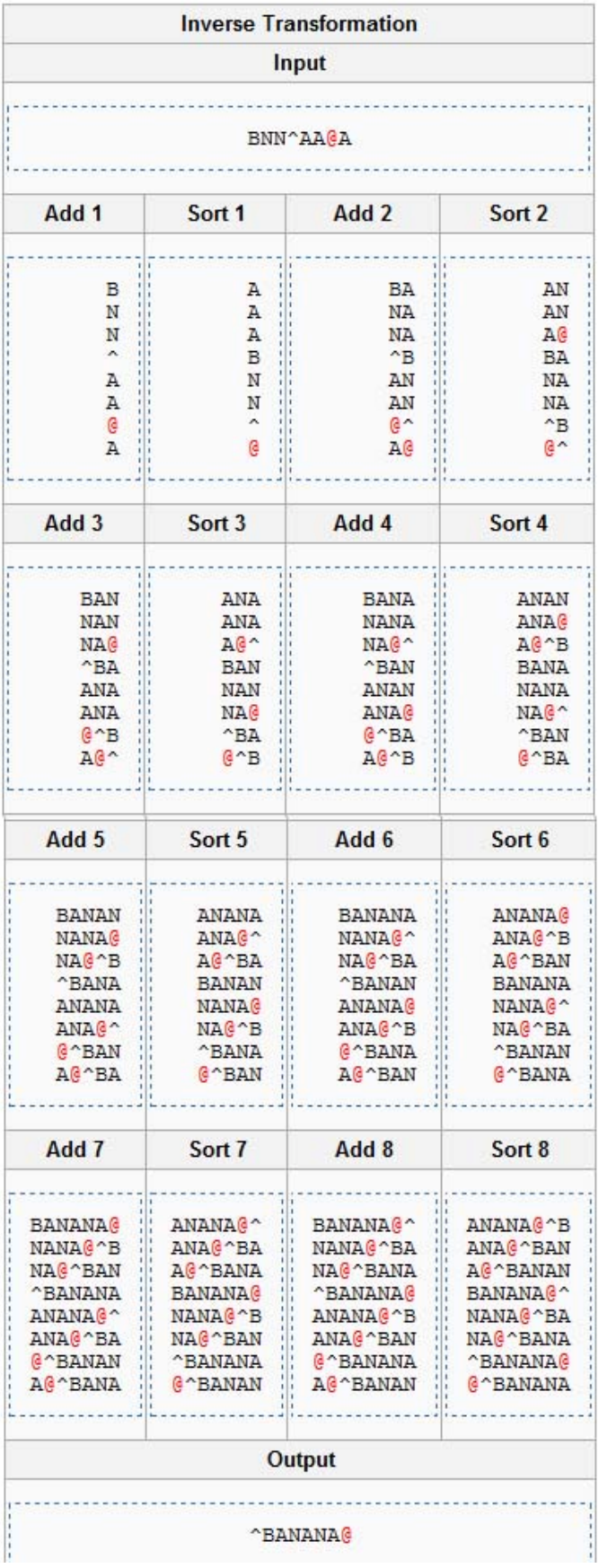
1. Transformasi Burrows Wheeler mampu menjadikan berkasyang memiliki karakter-karakter berulang dengan posisi yang berdekatan. Hal ini dapat menjadi jawaban atas kelemahan pada algoritma LZW.
2. Transformasi Burrows Wheeler sederhana untuk diimplementasikan karena tidak membutuhkan kamus tambahan pada proses transformasi. Pada transformasi Star maupun LIPT perlu dibangun sebuah kamus yang berisi kata-kata yang akan ditransformasi. Hal ini tentu akan menjadi kendala karena jumlah kata di dalam kamus dapat mencapai 400.000 buah kata.
3. Karena tidak menggunakan kamus tambahan, maka proses transformasi Burrows Wheeler tidak memiliki keterbatasan terhadap bahasa yang digunakan pada berkas, karena hanya melakukan permutasi terhadap isi berkas tersebut tanpa mencocokkan kata-kata berdasarkan kamus yang dibangun sebelumnya.

Untuk lebih memahami transformasi dan detransformasi Burrows Wheeler, diberikan contoh seperti pada gambar II-6 dan gambar II-7.

Transformation			
Input	All Rotations	Sorted List of Rotations	Output Last Column
^BANANA@	^BANANA@ @^BANANA A@^BANAN NA@^BANA ANA@^BAN NANA@^BA ANANA@^B BANANA@^	ANANA@^B ANA@^BAN A@^BANAN BANANA@^ NANA@^BA NA@^BANA ^BANANA@ @^BANANA	BNN^AA@A

Gambar II-6: Transformasi Burrows Wheeler

Pada gambar II-6, *string* "^BANANA@" akan mengalami rotasi sebanyak 8 kali. Rotasi yang terjadi ialah, memindahkan karakter paling terakhir menjadi paling awal pada *string*"^BANANA@". Setelah seluruh rotasi dilakukan, seluruh hasil rotasi akan diurutkan secara alfabetis naik. Hasil dari transformasi pada *string*"^BANANA@" adalah kolom terakhir pada matriks permutasi yang telah diurutkan, yaitu *string* "BNN^AA@A". Perlu diingat bahwa angka 7 sebagai penanda *string* awal pada matriks permutasi yang telah diurutkan perlu disimpan, karena akan digunakan pada detransformasi *string*.



Gambar II-7: Detransformasi Burrows Wheeler

Sedangkan pada proses detransformasi *string*, proses yang terjadi adalah menambahkan *string* hasil transformasi “BNN^AA@A” pada kolom paling kiri, kemudian mengurutkan secara alfabetis. Hal ini dilakukan secara berulang-ulang, yaitu 8 kali perulangan. Angka 7 yang telah disimpan akan digunakan untuk mengambil *string* awal pada matriks permutasi, yaitu baris ketujuh yang berisikan *string* awal “^BANANA@”.
Hasil dari transformasi kata “^BANANA@” adalah “BNN^AA@A”, dan disimpan angka 7, dimana 7 adalah nomor baris yang menyimpan kata “^BANANA@” pada matriks permutasi yang telah diurutkan sesuai alfabetis.

2.6 Pengembangan Perangkat Lunak

Proses pengembangan perangkat lunak merupakan struktur yang dikenakan di dalam pengembangan sebuah perangkat lunak. Proses pengembangan perangkat lunak juga disebut dengan siklus hidup perangkat lunak(Fischer, 2001). Proses pengembangan perangkat lunak dapat dibagi menjadi beberapa model pembangunan perangkat lunak. Model pembangunan perangkat lunak yang terkenal di antaranya ialah model air terjun, model spiral, model *iterative* dan *incremental*, serta model *agile*. Perangkat lunak yang akan dibangun pada tugas akhir ini akan menggunakan model air terjun.

Aktivitas utama pada proses pengembangan perangkat lunak diantaranya ialah perencanaan, implementasi, evaluasi, dokumentasi, penyebaran (*deployment*) dan pengawasan (*maintaining*). Perencanaan merupakan tahapan awal dalam pengembangan perangkat lunak. Pada tahap perencanaan, tujuan utama ialah mengekstraksi kebutuhan atau analisis kebutuhan perangkat lunak dari klien. Hal penting dari tahap ini juga ialah pembatasan pengembangan atau disebut juga analisis *scope* perangkat lunak, yaitu sampai sejauh apa perangkat lunak akan dikembangkan.

Implementasi adalah bagian dari proses dimana para *software engineer* melakukan *coding* terhadap proyek yang diberikan. Evaluasi adalah tahap yang sangat penting dari pengembangan perangkat lunak, yaitu memastikan bahwa perangkat lunak terbebas dari *bug* dan *error*. Sedangkan dokumentasi berguna untuk pengawasan di masa depan serta pengembangan perangkat lunak di kemudian hari. Penulisan API

(*Application Programming Interface*) juga termasuk di dalam proses dokumentasi. Dokumentasi setiap kejadian dan sesuatu di dalam proyek sangatlah dianjurkan.

Penyebaran perangkat lunak akan dimulai setelah *code* dari program sudah dievaluasi dengan semestinya.Pengawasan perangkat lunak dapat menyita waktu jauh lebih lama dibandingkan pembangunan perangkat lunak. Pada pengawasan, klien dapat meminta pembuatan *code* baru dikarenakan fungsionalitas perangkat lunak tidak berjalan sesuai desain perangkat lunak pada tahap perencanaan. Biaya pengawasan perangkat lunak adalah yang paling besar di dalam anggaran pengembangan perangkat lunak. Sebuah perangkat lunak yang baik tidak boleh memiliki biaya pengawasan yang melebihi 25% biaya pembangunan perangkat lunak tersebut.

BAB III
DESKRIPSI SOLUSI

Bab ini akan membahas deskripsi solusi dari masalah yaitu bagaimana kinerja kompresi LZW dapat ditingkatkan melalui transformasi Burrows Wheeler. Selain diberikan analisis terhadap solusi yang diusulkan, bab ini juga akan membahas tentang analisis,perancangan, serta implementasiperangkat lunak yang akan dibuat, antara lain mengenai diagram kelas analisis, perancangan *use case* tahap analisis, diagram kelas perancangan, dan perancangan *use case* tahap perancangan.

3.1 Kompresi Algoritma Lempel Ziv Welch

Algoritma Lempel Ziv Welch, atau selanjutnya disebut dengan algoritma LZW merupakan algoritma kompresi yang cukup populer. Proses kompresi maupun dekompresi algoritma LZW secara spesifik akan dijabarkan pada pembahasan berikut. Kompresi algoritma LZW dimulai dengan menginisialisasi kamus dengan seluruh simbol di dalam alfabet. Jumlah keseluruhan simbol adalah 256 buah, sehingga pada mulanya kamus algoritma LZW telah terisi dengan 256 buah simbol sebelum dilakukan pembacaan pada berkas yang akan dikompresi. Hal inilah yang merupakan keunggulan utama pada algoritma LZW, yaitu karena kamus sudah diisi terlebih dahulu, maka karakter yang akan dibaca pada berkas pastilah telah terdapat di dalam kamus, sehingga token LZW hanyalah mengandung pointer karakter tersebut pada kamus saja. Akibatnya, berkas hasil kompresi akan menjadi lebih mampat dibandingkan hasil kompresi algoritma lain seperti LZ77.

Prinsip utama proses kompresi algoritma LZW adalah membaca setiap karakter satu per satu dan mengakumulasikannya ke dalam sebuah string. Selama string ini masih ditemukan di dalam kamus, maka proses pembacaan akan terus berlanjut. Ketika string tidak ditemukan di dalam kamus, sebut saja string Sx (yaitu string S dikonkatenasi dengan karakter x), maka urutan string S dalam kamus akan dituliskan oleh encoder LZW, string Sx akan ditambahkan pada kamus, dan menginisialisasi string awal sebagai karakter x.

Untuk memahami secara mendetail, diberikan sebuah contoh kasus kompresi teks “botbananaboat”. Karakter yang dibaca pertama kali ialah “b”, yaitu terdapat di dalam kamus dengan indeks 98. Selanjutnya “b” akan dikonkatenasi dengan “o”, dimana “bo” tidak terdapat di dalam kamus. Sehingga encoder LZW akan menuliskan 98, yaitu indeks karakter “b”, kemudian menambahkan “bo” di dalam kamus, dan mulai melakukan pembacaan karakter dari karakter “o”.

Proses kompresi tahap per tahap dari teks “botbananaboat” ditunjukkan pada tabel berikut.

Tabel III-1: Kompresi Teks "botbananaboat"

String Awal	Di dalam kamus?	Entri baru di dalam kamus	Keluaran encoder
b	Ada		
bo	Tidak	bo(257)	b
o	Ada		
ot	Tidak	ot(258)	o
t	Ada		
tb	Tidak	tb(259)	t
b	Ada		
ba	Tidak	ba(260)	b(98)
a	Ada		
an	Tidak	an(261)	a(97)
n	Ada		
na	Tidak	na(262)	n(110)
a	Ada		
an	Ada		
ana	Tidak	ana(263)	an(258)
a	Ada		
ab	Tidak	ab(264)	a(97)
b	Ada		
bo	Ada		
boa	Tidak	boa(265)	bo
a	Ada		
at	Tidak	at(266)	a(97)

String Awal	Di dalam kamus?	Entri baru di dalam kamus	Keluaran encoder
t	Ada		
t,EOF	Tidak		t(116)

Maka hasil dari kompresi teks “botbananaboat” dengan algoritma LZW adalah b(98), o(111), t(116), b(98), a(97), n(110), an(258), a(97), bo(256), a(97), t(116).

3.2 Kelemahan Algoritma LZW

Algoritma LZW cukup mangkus dalam melakukan kompresi data, namun algoritma LZW sendiri memiliki kelemahan, diantaranya ialah:

Waktu kompresi yang semakin lama apabila jumlah entri dalam kamus semakin besar. Kompresi algoritma LZW dilakukan dengan mengkompresi karakter per karakter, dimana setiap karakter akan dicek apakah telah berada di dalam kamus atau tidak. Waktu yang diperlukan untuk mengecek keberadaan setiap karakter ini akan semakin lama apabila entri dalam kamus semakin banyak. Hal ini dikarenakan kamus akan ditelusuri dari entri pertama hingga entri terakhir. Contohnya: apabila entri terakhir memiliki indeks 875, maka iterasi yang dibutuhkan adalah 875 kali iterasi, yang tentunya akan memakan waktu yang cukup lama.

Pada berkas dengan karakter-karakter yang tersebar acak, jumlah entri dalam kamus akan menjadi sangat besar, dikarenakan string yang dimasukkan di dalam kamus tidak memiliki keterkaitan dengan string lainnya yang telah berada di dalam kamus. Dengan kata lain, kompresi berkas malah mengakibatkan berkas hasil kompresi dapat berukuran lebih besar dibandingkan ukuran berkas sebelum dikompresi.

Berdasarkan kedua kelemahan tersebut, maka permasalahan utama dari algoritma LZW ialah jumlah entri dalam kamus yang sangat besar, sehingga solusi yang terbaik untuk menyelesaikan persoalan ini ialah mengubah bentuk berkas sebelum dikompresi menjadi lebih teratur sehingga ukuran kamus dapat diperkecil.

3.3 Solusi yang Ditawarkan

Pengubahan bentuk berkas dapat dilakukan dengan berbagai macam metode. Salah satu metode yang terkenal adalah transformasi berkas. Transformasi berkas yang digunakan pada pengerjaan tugas akhir ini ialah transformasi Burrows Wheeler.

Transformasi Burrows Wheeler akan mengubah bentuk berkas masukan, sehingga karakter-karakter yang sejenis akan berada di dalam posisi berdekatan di dalam berkas. Transformasi Burrows Wheeler menggunakan sistem permutasi pada setiap karakter di dalam berkas.

Pengubahan bentuk berkas tidak akan menggunakan transformasi Star dan LIPT. Alasannya ialah kedua bentuk transformasi ini tidak efisien karena memerlukan kamus tambahan di dalam proses transformasi. Kamus tambahan yang dimaksud ialah daftar seluruh kata-kata yang terdapat di dalam bahasa (sebagai contoh bahasa Inggris apabila berkas disadurkan dalam bahasa Inggris). Kesulitan dalam pembuatan kamus tambahan inilah yang menyebabkan transformasi Star dan LIPT tidak digunakan dalam pembuatan tugas akhir ini.

Transformasi Burrows Wheeler akan digunakan pada tugas akhir ini karena transformasi ini bersifat sederhana dan tidak memerlukan kamus tambahan. Transformasi Burrows Wheeler dapat mengubah berkas sehingga karakter-karakter yang sama akan diletakkan di dalam posisi yang berdekatan di dalam berkas.

3.4 Analisis Solusi

Tinjau kembali teks “botbananaboat” yang telah dibahas pada subbab 3.1. Teks ini akan mengalami proses transformasi Burrows Wheeler sehingga akan menghasilkan berkas yang berbeda, misalnya menjadi:

B	B	B	A	A	A	A	N	N	O	O	T	T

Hasil dari transformasi Burrows Wheeler terhadap teks “botbananaboat” adalah “bbbaaaannoott”. Perhatikan bahwa karakter-karakter yang sama pada teks hasil transformasi akan menjadi saling berdekatan. Setelah itu teks “nnbotaaaba” akan dikompresi dengan algoritma LZW.

Tabel III-2: Kompresi Teks Hasil Transformasi

String Awal	Di dalam kamus?	Entri baru di dalam kamus	Keluaran encoder
b	Ada		
bb	Tidak	bb(257)	b
b	Ada		
bb	Ada		
bba	Tidak	bba(258)	bb
a	Ada		
aa	Tidak	aa(259)	a
a	Ada		
aa	Ada		
aaa	Tidak	aaa(260)	aa
a	Ada		
an	Tidak	an(261)	a
n	Ada		
nn	Tidak	nn(262)	n
n	Ada		
no	Tidak	no(263)	n
o	Ada		
ot	Tidak	ot(264)	o
t	Ada		
tt	Tidak	tt(265)	t
t	Ada		
t, EOF	Tidak		t

Hasil kompresi berkas “botbananaboat” akan menjadi lebih baik apabila berkas ditransformasikan dengan transformasi Burrows Wheeler. Kompresi akan semakin baik apabila jumlah huruf yang berulang semakin banyak, misalnya “maaaaaa” akan memberikan kompresi yang lebih baik daripada “maa”.

3.5 Faktor-Faktor yang Menentukan

Beberapa faktor yang menentukan tingkat keberhasilan algoritma kompresi LZW dengan transformasi berkas Burrows Wheeler antara lain:

Bentuk transformasi berkas

Secara teori berkas yang telah tertransformasi akan semakin baik hasil kompresinya, karena karakter-karakter yang berulang telah berada dalam posisi yang berdekatan. Namun, ada kalanya pola repetisi pada berkas sudah baik sehingga transformasi berkas justru dapat membuat bentuk berkas menjadi lebih rumit, yang akibatnya akan sulit untuk dikompresi.

Ukuran kamus

Ukuran kamus pada algoritma LZW juga memiliki peranan penting. Semakin besar ukuran kamus maka waktu kompresi yang dibutuhkan akan semakin lama. Oleh karena itu, transformasi Burrows Wheeler digunakan untuk memperkecil ukuran kamus, sehingga diharapkan waktu kompresi akan menjadi lebih singkat.

3.6 Analisis Perangkat Lunak

3.6.1 Deskripsi Perangkat Lunak

Perangkat lunak yang akan dibangun ialah perangkat lunak yang mampu melakukan pengujian terhadap hasil kompresi berkas berjenis teks dengan algoritma LZW. Selain itu, perangkat lunak yang akan dibangun harus mampu untuk mentransformasikan berkas berjenis teks dengan transformasi Burrows Wheeler. Perangkat lunak yang akan dibangun merupakan perangkat lunak yang sederhana, karena tujuan dari pembangunan perangkat lunak ialah untuk menguji hasil kompresi berkas sebelum dan sesudah ditransformasikan dengan transformasi Burrows Wheeler.

Berkas yang ditangani perangkat lunak adalah berkas berjenis *plaintext* (*.txt). Perangkat lunak yang akan dibangun memiliki antarmuka yang memudahkan pengguna untuk melakukan proses kompresi maupun transformasi pada berkas. Antarmuka akan dirancang menjadi sesederhana mungkin namun tepat guna.

3.6.2 Spesifikasi Kebutuhan Perangkat Lunak

Kebutuhan fungsional perangkat lunak dapat dilihat pada tabel III-3.

Tabel III-3: Daftar Kebutuhan Fungsional

ID	Deskripsi Kebutuhan
SRS-F-001	Sistem mampu melakukan kompresi berkas masukan berjenis teks dengan algoritma LZW
SRS-F-002	Sistem mampu melakukan dekompresi berkas masukan hasil kompresi dengan algoritma LZW
SRS-F-003	Sistem mampu melakukan kompresi berkas masukan berjenis teks dengan gabungan algoritma LZW dan Burrows Wheeler
SRS-F-004	Sistem mampu melakukan dekompresi berkas masukan hasil kompresi dengan gabungan algoritma LZW dan Burrows Wheeler

Kebutuhan non-fungsional perangkat lunak dapat dilihat pada tabel III-4.

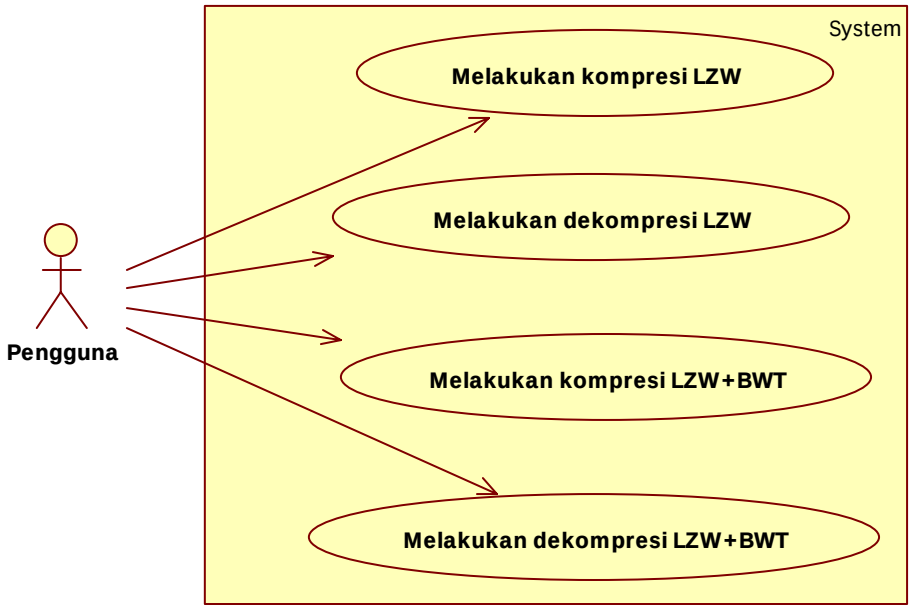
Tabel III-4: Daftar Kebutuhan Non Fungsional

ID	Deskripsi Kebutuhan
SRS-NF-001	Sistem memiliki antarmuka yang mudah dimengerti oleh pengguna
SRS-NF-002	Sistem ditampilkan dalam bahasa Indonesia

3.6.3 Diagram *Use Case*

Diagram *use case* dari perangkat lunak dapat dilihat pada gambar III-1. Di dalam perangkat lunak ini terdapat seorang aktor, yaitu pengguna. *Use case* di dalam perangkat lunak ini adalah:

1. Melakukan kompresi berkas dengan algoritma LZW.
2. Melakukan dekompresi berkas dengan algoritma LZW.
3. Melakukan kompresi berkas dengan algoritma LZW+BWT.
4. Melakukan dekompresi berkas dengan algoritma LZW+BWT.



Gambar III-1: Diagram *Use Case*

3.6.4 Skenario *Use Case*

ID : UC-01

Nama *use case* : Melakukan kompresi berkas

Deskripsi : Pengguna dapat melakukan kompresi berkas dengan algoritma LZW

Prekondisi : Pengguna memilih berkas yang akan dikompresi

Tabel III-5: Skenario *Use Case* 01

Aksi Aktor	Reaksi Sistem
1. Mengklik tombol untuk memilih berkas yang akan dikompresi	
	2. Menampilkan *open file dialog*
3. Mengklik tombol untuk memilih berkas hasil kompresi	
	4. Menampilkan *save file dialog*
5. Memilih algoritma LZW, memilih mode kompresi	
6. Mengklik eksekusi	

Aksi Aktor	Reaksi Sistem
	7. Melakukan kompresi pada berkas dengan algoritma LZW
	8. Memberitahukan kepada pengguna bahwa proses kompresi telah selesai

ID : UC-02

Nama *use case* : Melakukan dekompresi berkas

Deskripsi : Pengguna dapat melakukan dekompresi berkas dengan algoritma LZW

Prekondisi : Pengguna memilih berkas yang akan didekompresi

Tabel III-6: Skenario *Use Case* 02

Aksi Aktor	Reaksi Sistem
1. Mengklik tombol untuk memilih berkas yang akan dikompresi	
	2. Menampilkan *open file dialog*
3. Mengklik tombol untuk memilih berkas hasil kompresi	
	4. Menampilkan *save file dialog*
5. Memilih algoritma LZW, memilih mode dekompresi	
6. Mengklik eksekusi	
	7. Melakukan kompresi pada berkas dengan algoritma LZW
	8. Memberitahukan kepada pengguna bahwa proses kompresi telah selesai

ID : UC-03

Nama *use case* : Melakukan kompresi berkas dengan algoritma gabungan

Deskripsi : Pengguna dapat melakukan kompresi berkas dengan algoritma gabungan LZW dan Burrows Wheeler

Prekondisi : Pengguna memilih berkas yang akan dikompresi

Tabel III-7: Skenario *Use Case* 03

Aksi Aktor	Reaksi Sistem
1. Mengklik tombol untuk memilih berkas yang akan dikompresi	
	2. Menampilkan *open file dialog*
3. Mengklik tombol untuk memilih berkas hasil kompresi	
	4. Menampilkan *save file dialog*
5. Memilih algoritma LZW + BWT, memlih mode kompresi	
6. Mengklik eksekusi	
	7. Melakukan transformasi berkas dengan algoritma Burrows Wheeler
	8. Melakukan kompresi berkas dengan algoritma LZW
	9. Memberitahukan kepada pengguna bahwa proses kompresi telah selesai

ID : UC-04

Nama *use case* : Melakukan dekompresi berkas dengan algoritma gabungan

Deskripsi : Pengguna dapat melakukan dekompresi berkas dengan algoritma gabungan LZW dan Burrows Wheeler

Prekondisi : Pengguna memilih berkas yang akan didekompresi

Tabel III-8: Skenario *Use Case* 04

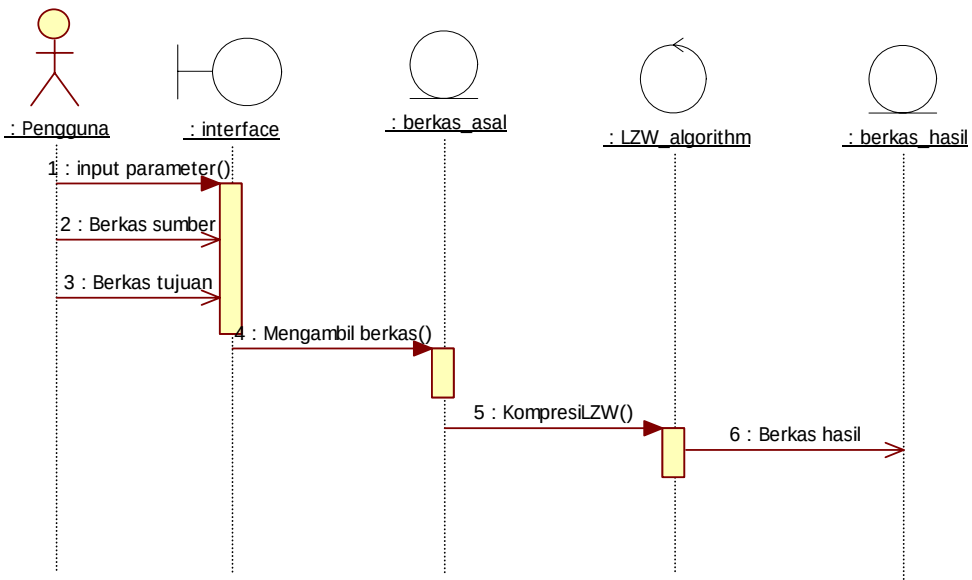
Aksi Aktor	Reaksi Sistem
1. Mengklik tombol untuk memilih berkas yang akan didekompresi	
	2. Menampilkan *open file dialog*
3. Mengklik tombol untuk memilih berkas hasil dekompresi	
	4. Menampilkan *save file dialog*
5. Memilih algoritma LZW + BWT, memlih mode dekompresi	
6. Mengklik eksekusi	

Aksi Aktor	Reaksi Sistem
	7. Melakukan dekompresi berkas dengan algoritma LZW
	8. Melakukan detransformasi berkas dengan algoritma Burrows Wheeler
	9. Memberitahukan kepada pengguna bahwa proses dekompresi telah selesai

3.6.5 Realisasi *Use Case* Tahap Analisis

3.6.5.1 Use Case Melakukan Kompresi Berkas

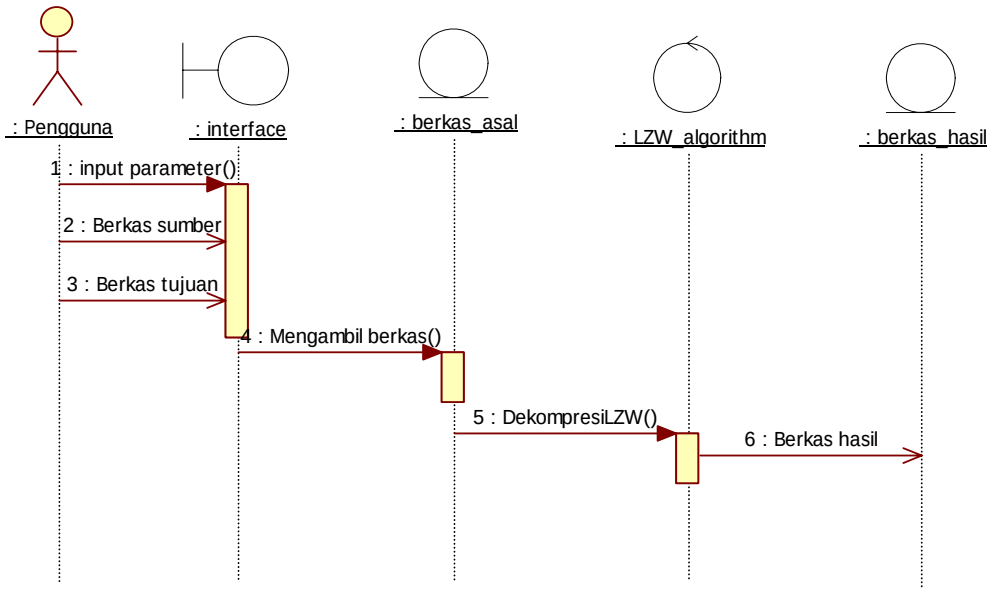
Pada *use case* melakukan kompresi berkas, terdapat empat buah kelas yang terlibat. Kelas interface adalah antarmuka yang digunakan oleh pengguna. Kelas LZW_algorithm adalah kelas *controller* yang menangani kompresi berkas dengan algoritma LZW. Kelas berkas_asal merupakan kelas *entity* yang berisikan berkas yang akan dikompresi, dan kelas berkas_hasil merupakan kelas *entity* yang berisikan berkas yang hasil kompresi.



Gambar III-2: *Sequence Diagram* Analisis *Use Case* 01

3.6.5.2 *Use Case Melakukan Dekompresi Berkas*

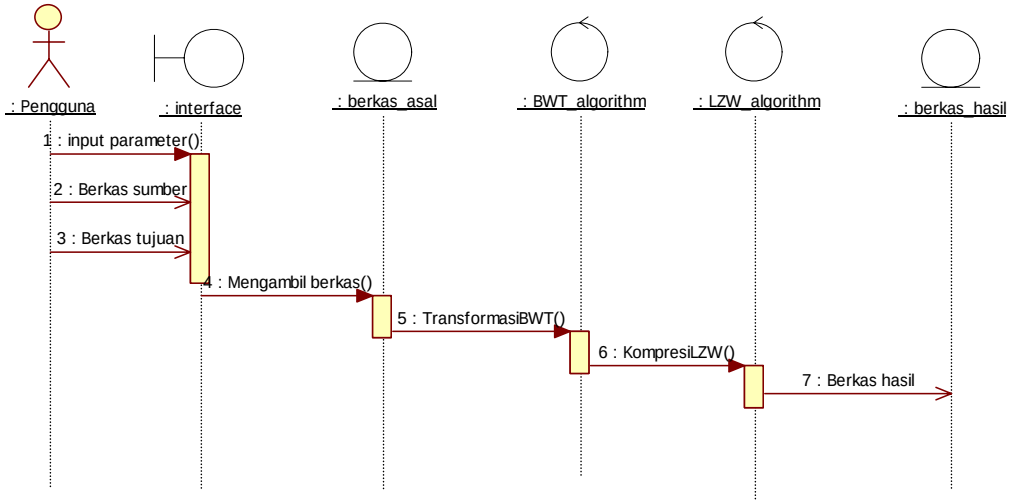
Pada *use case* melakukan dekompresi berkas, terdapat empat buah kelas yang terlibat. Kelas interface adalah antarmuka yang digunakan oleh pengguna. Kelas LZW_algorithm adalah kelas *controller* yang menangani dekompresi berkas dengan algoritma LZW. Kelas berkas_asal merupakan kelas *entity* yang berisikan berkas yang akan didekompresi, dan kelas berkas_hasil merupakan kelas *entity* yang berisikan berkas hasil dekompresi.



Gambar III-3: *Sequence Diagram* Analisis *Use Case* 02

3.6.5.3 *Use Case Melakukan Kompresi Berkas dengan Algoritma Gabungan*

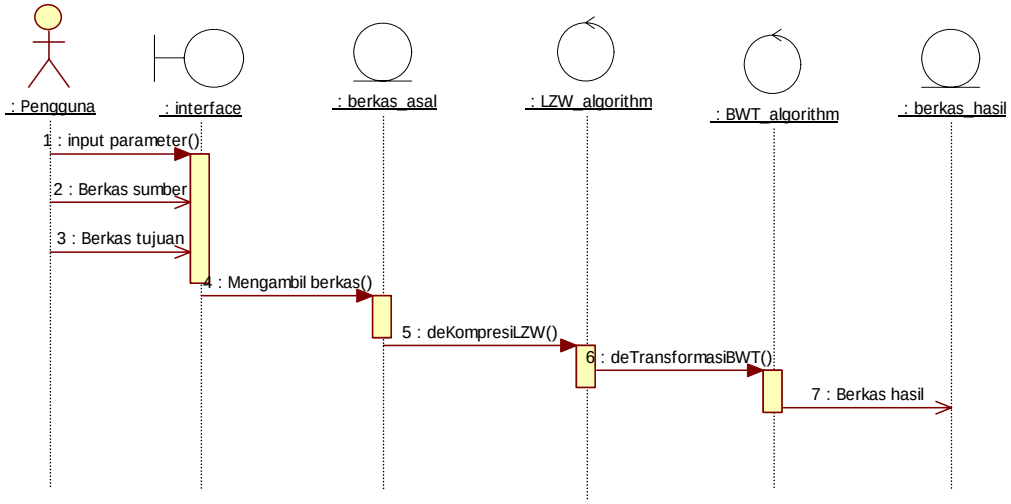
Pada *use case* melakukan kompresi berkas dengan algoritma gabungan, terdapat lima buah kelas yang terlibat. Kelas interface adalah antarmuka yang digunakan oleh pengguna. Kelas LZW_algorithm adalah kelas *controller* yang menangani kompresi berkas dengan algoritma LZW. Kelas BWT_algorithm adalah kelas *controller* yang menangani transformasi berkas dengan transformasi Burrows Wheeler. Kelas berkas_asal merupakan kelas *entity* yang berisikan berkas yang akan dikompresi, dan kelas berkas_hasil merupakan kelas *entity* yang berisikan berkas hasil kompresi.



Gambar III-4: *Sequence Diagram* Analisis *Use Case* 03

3.6.5.4 Use Case Melakukan Dekompresi Berkas dengan Algoritma Gabungan

Pada *use case* melakukan kompresi berkas dengan algoritma gabungan, terdapat lima buah kelas yang terlibat. Kelas interface adalah antarmuka yang digunakan oleh pengguna. Kelas LZW_algorithm adalah kelas *controller* yang menangani dekompresi berkas dengan algoritma LZW. Kelas BWT_algorithm adalah kelas *controller* yang menangani detransformasi berkas dengan transformasi Burrows Wheeler. Kelas berkas_asal merupakan kelas *entity* yang berisikan berkas yang akan didekompresi, dan kelas berkas_hasil merupakan kelas *entity* yang berisikan berkas hasil dekompresi.

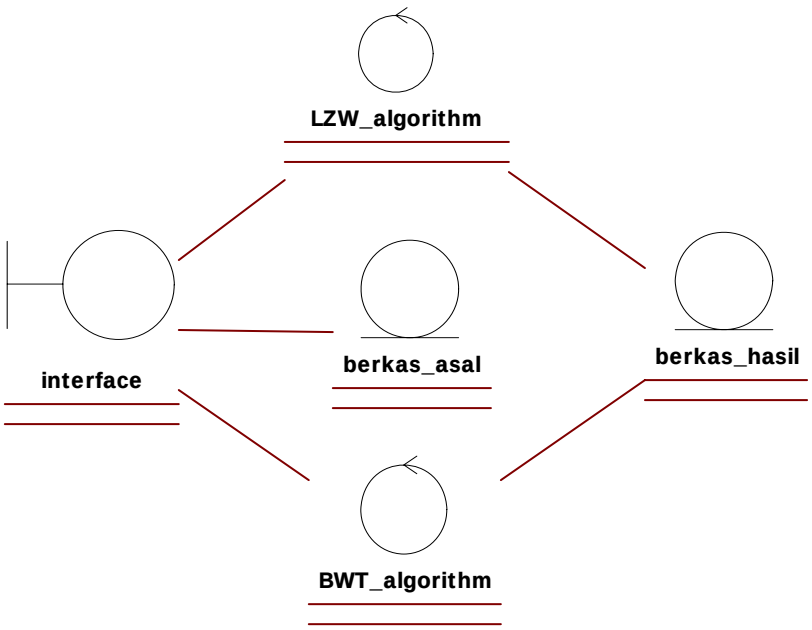


Gambar III-5: *Sequence Diagram* Analisis *Use Case* 04

3.6.6 Diagram Kelas Analisis

Diagram kelas analisis perangkat lunak dapat dilihat pada gambar III-6. Kelas-kelas yang terlibat antara lain:

1. Kelas interface, adalah kelas *boundary* yang merupakan antarmuka untuk aktor pengguna.
2. Kelas BWTAlgorithm, adalah kelas *controller* yang merupakan kelas yang menangani proses transformasi dan detransformasi Burrows Wheeler pada berkas.
3. Kelas LZWAlgorithm, adalah kelas *controller* yang merupakan kelas yang menangani proses kompresi dan dekompresi LZW pada berkas.
4. Kelas berkas_asal, adalah kelas *entity* yang merupakan berkas yang akan dikompresi, didekompresi, ditransformasi, dan didetransformasi.
5. Kelas berkas_hasil, adalah kelas *entity* yang merupakan berkas hasil kompresi, dekompresi, transformasi, maupun detransformasi.

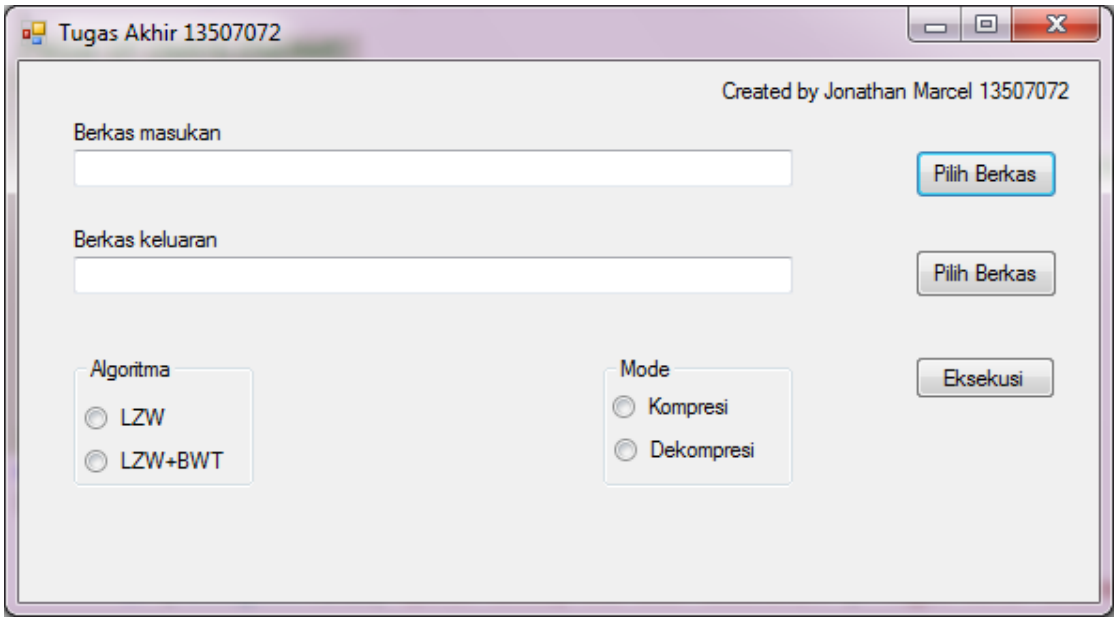


Gambar III-6: Diagram Kelas Analisis

3.7 Perancangan Perangkat Lunak

3.7.1 Perancangan Antarmuka

Perangkat lunak akan memiliki tampilan seperti pada gambar berikut. Pengguna dapat memilih berkas asal yang diinginkan dan dapat menyimpan berkas hasil dengan nama berkas yang diinginkan. Pengguna juga dapat memilih pilihan algoritma yang akan digunakan, yaitu kompresi, dekompresi, kompresi gabungan, dan dekompresi gabungan dengan mengklik tombol yang sesuai.



Gambar III-7: Antarmuka Utama Perangkat Lunak

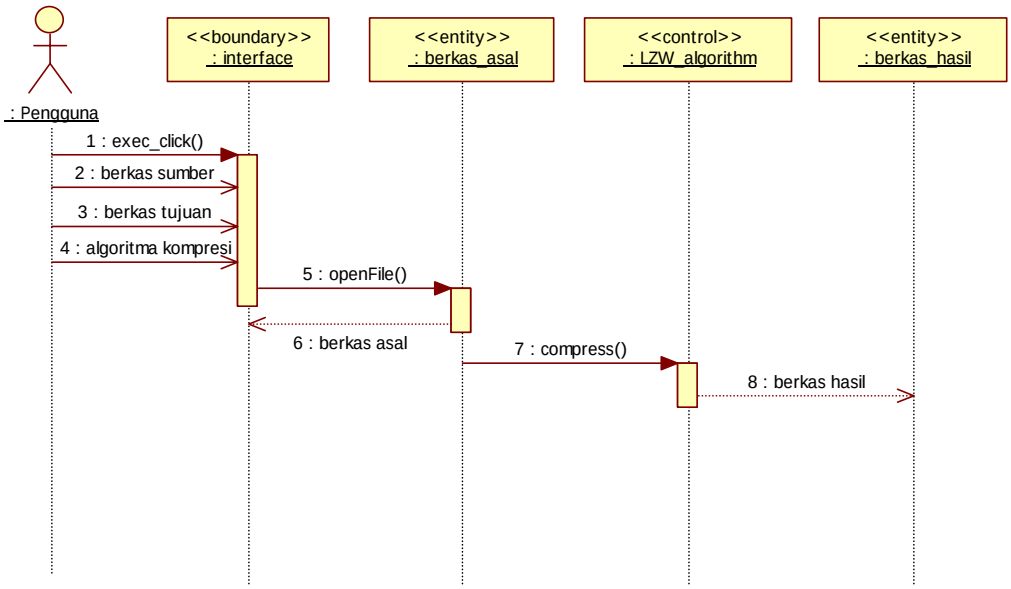
Setelah proses kompresi, dekompresi, kompresi gabungan, maupun dekompresi gabungan selesai, sistem akan menampilkan *message box* yang berisikan informasi bahwa proses telah selesai.

3.7.2 Realisasi *Use Case* Tahap Perancangan

3.7.2.1 Use Case Melakukan Kompresi Berkas

Pada *use case* melakukan kompresi berkas, terdapat empat buah kelas yang terlibat. Kelas interface adalah antarmuka yang digunakan oleh pengguna. Kelas LZW_algorithm adalah kelas *controller* yang menangani kompresi berkas dengan algoritma LZW. Kelas berkas_asal merupakan kelas *entity* yang berisikan berkas yang

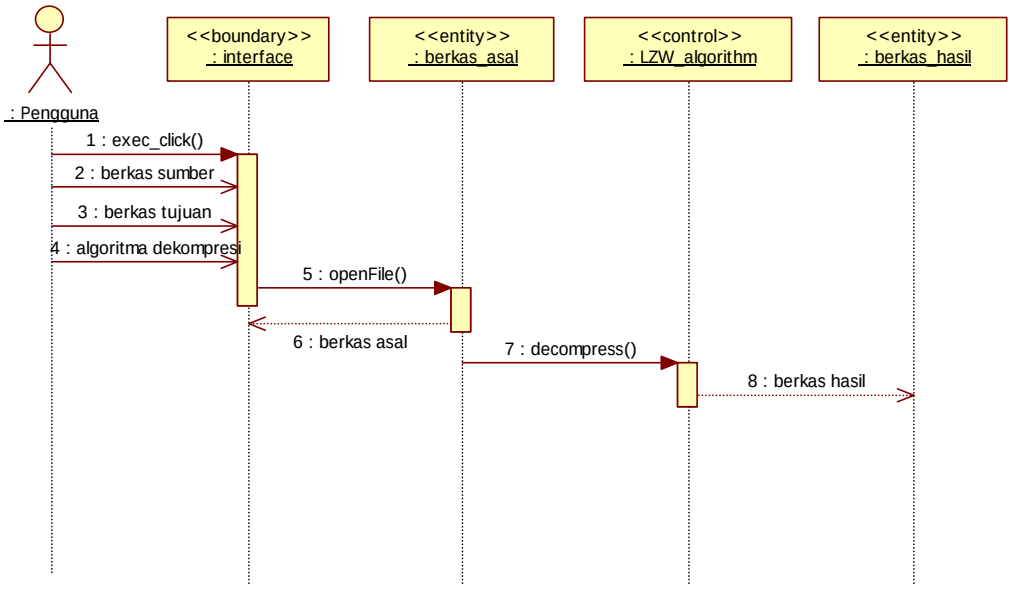
akan dikompresi, dan kelas berkas_hasil merupakan kelas *entity* yang berisikan berkas yang hasil kompresi.



Gambar III-8: *Sequence Diagram* Perancangan *Use Case* 01

3.7.2.2 Use Case Melakukan Dekompresi Berkas

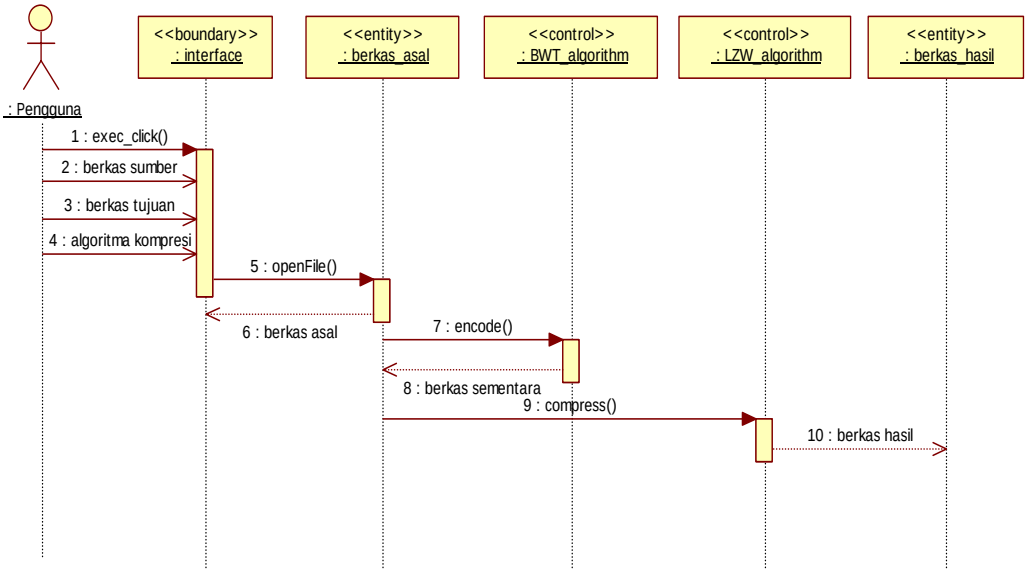
Pada *use case* melakukan dekompresi berkas, terdapat empat buah kelas yang terlibat. Kelas interface adalah antarmuka yang digunakan oleh pengguna. Kelas LZW_algorithm adalah kelas *controller* yang menangani dekompresi berkas dengan algoritma LZW. Kelas berkas_asal merupakan kelas *entity* yang berisikan berkas yang akan didekompresi, dan kelas berkas_hasil merupakan kelas *entity* yang berisikan berkas yang hasil dekompresi.



Gambar III-9: *Sequence Diagram* Perancangan *Use Case* 02

3.7.2.3 Use Case Melakukan Kompresi Berkas dengan Algoritma Gabungan

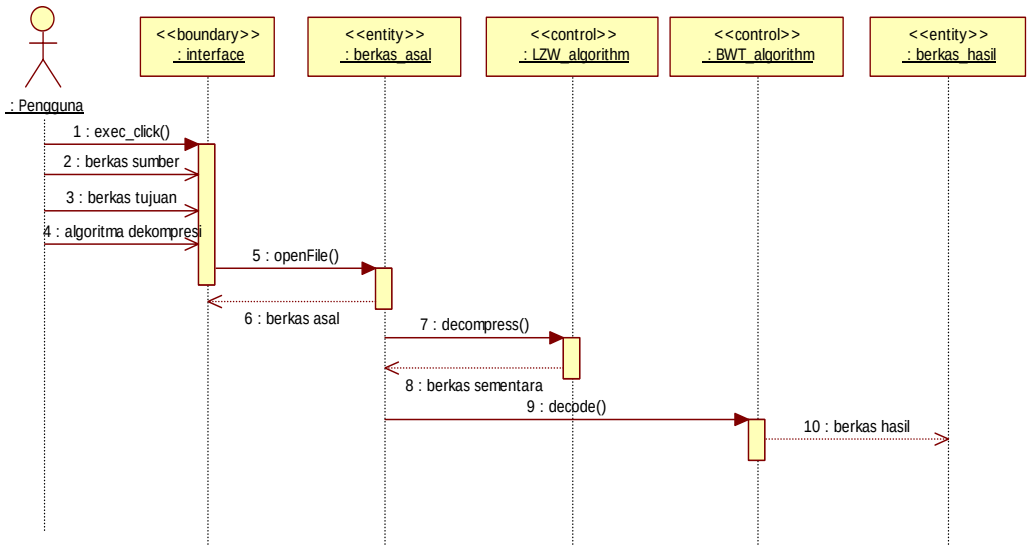
Pada *use case* melakukan kompresi berkas dengan algoritma gabungan, terdapat lima buah kelas yang terlibat. Kelas interface adalah antarmuka yang digunakan oleh pengguna. Kelas LZW_algorithm adalah kelas *controller* yang menangani kompresi berkas dengan algoritma LZW. Kelas BWT_algorithm adalah kelas *controller* yang menangani transformasi berkas dengan transformasi Burrows Wheeler. Kelas berkas_asal merupakan kelas *entity* yang berisikan berkas yang akan dikompresi, dan kelas berkas_hasil merupakan kelas *entity* yang berisikan berkas hasil kompresi.



Gambar III-10: *Sequence Diagram* Perancangan *Use Case* 03

3.7.2.4 Use Case Melakukan Dekompresi Berkas dengan Algoritma Gabungan

Pada *use case* melakukan kompresi berkas dengan algoritma gabungan, terdapat lima buah kelas yang terlibat. Kelas interface adalah antarmuka yang digunakan oleh pengguna. Kelas LZW_algorithm adalah kelas *controller* yang menangani dekompresi berkas dengan algoritma LZW. Kelas BWT_algorithm adalah kelas *controller* yang menangani detransformasi berkas dengan transformasi Burrows Wheeler. Kelas berkas_asal merupakan kelas *entity* yang berisikan berkas yang akan didekompresi, dan kelas berkas_hasil merupakan kelas *entity* yang berisikan berkas hasil dekompresi.

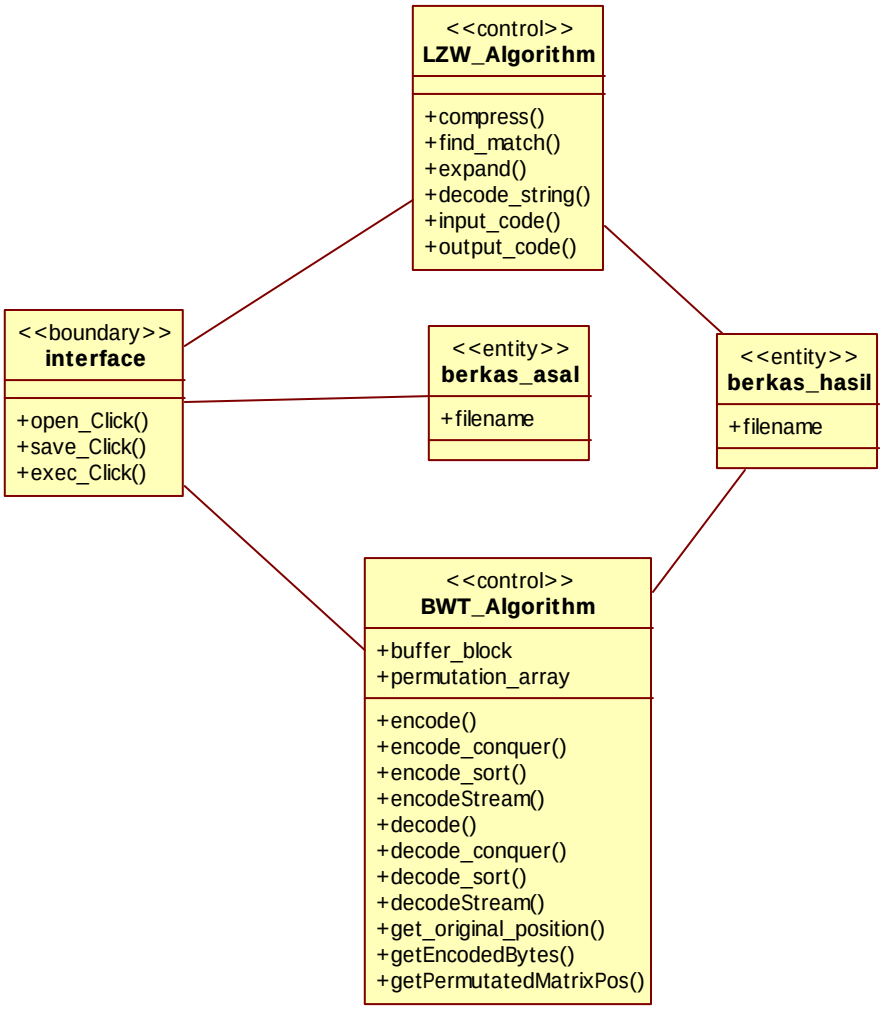


Gambar III-11: *Sequence Diagram* Perancangan *Use Case* 04

3.7.3 Diagram Kelas Perancangan

Diagram kelas perancangan perangkat lunak dapat dilihat pada gambar III-12. Kelas-kelas yang terlibat antara lain:

1. interface, *boundaryclass*, yang merupakan antarmuka untuk aktor pengguna.
2. BWT_Algorithm, *controller class*, yang merupakan kelas yang menangani proses transformasi dan detransformasi Burrows Wheeler pada berkas.
3. LZW_Algorithm, *controller class*, yang merupakan kelas yang menangani proses kompresi dan dekompresi LZW pada berkas.
4. berkas_asal, *entity class*, yang merupakan berkas yang akan dikompresi, didekompresi, ditransformasi, dan didetransformasi.
5. berkas_hasil, *entity class*, yang merupakan berkas hasil kompresi, dekompresi, transformasi, maupun detransformasi.



Gambar III-12: Diagram Kelas Perancangan

3.8 Implementasi Perangkat Lunak

3.8.1 Lingkungan Implementasi

Perangkat lunak dikembangkan pada perangkat keras berbasis *desktop* dengan spesifikasi sebagai berikut:

1. Prosesor Intel Core i5 2,26 GHz
2. Memory RAM 2GB
3. Harddisk 500GB
4. Perangkat masukan *keyboard* dan *mouse*

Adapun perangkat lunak yang digunakan memiliki spesifikasi sebagai berikut:

1. Sistem operasi Microsoft Windows 7

2. Kakas pengembang Microsoft Visual Studio 2010 dengan bahasa pemrograman C#
3. Notepad++

3.8.2 Batasan Implementasi

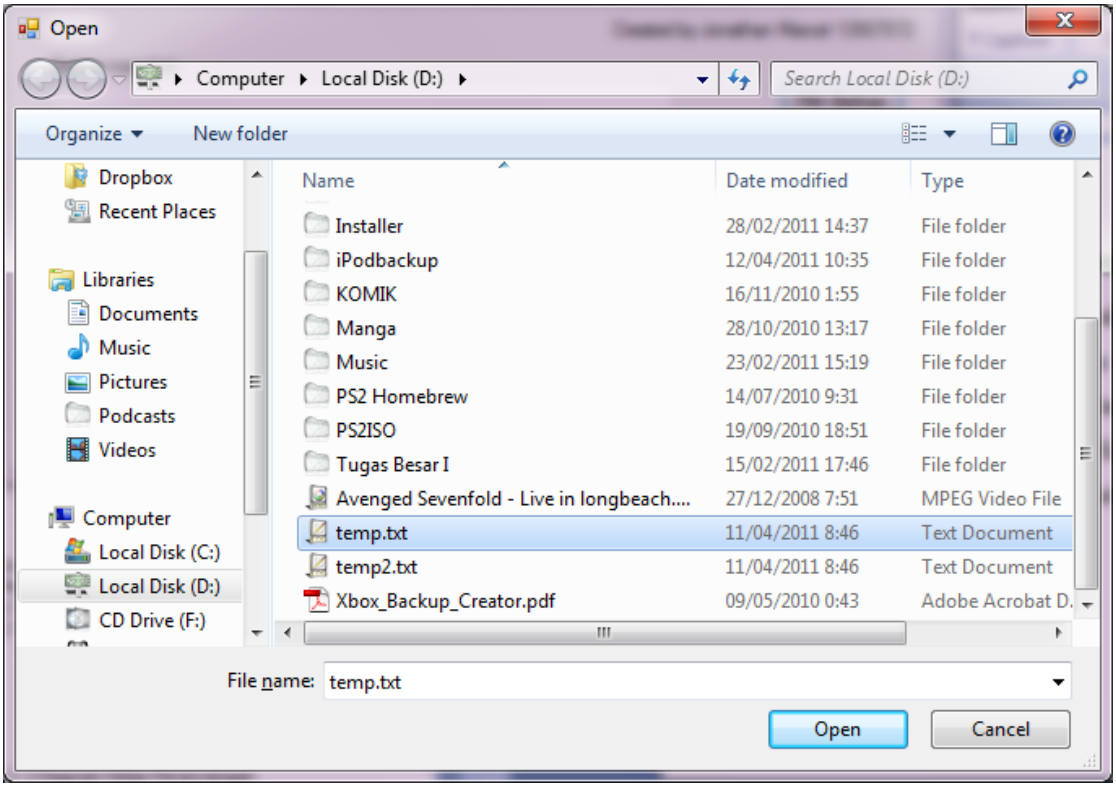
Perangkat lunak hanya akan memroses berkas berjenis *plain text* dengan ekstensi (*.txt), dan terdapat validasi terhadap berkas masukan pengguna.

3.8.3 Implementasi Antarmuka

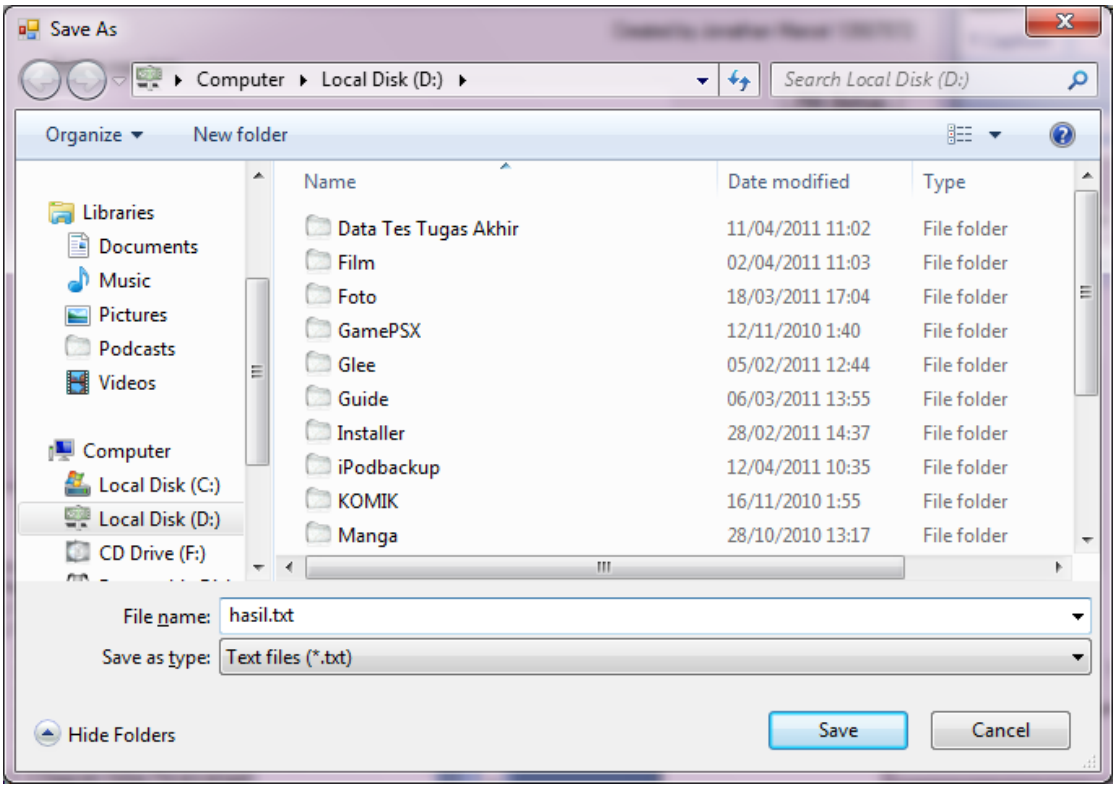
Implementasi antarmuka perangkat lunak sesuai dengan perancangan antarmuka pada subbab 3.7.1, dengan tambahan *open file dialog*, *save file dialog*, dan *message box*. *Open file dialog* akan muncul ketika pengguna memilih berkas yang akan dimanipulasi. *Save file dialog* akan muncul ketika pengguna akan menyimpan berkas hasil manipulasi.

Message Box akan ditampilkan kepada pengguna apabila:

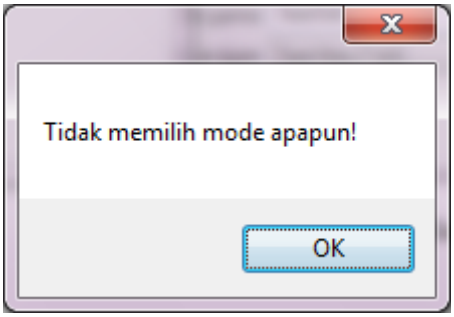
1. Pengguna tidak memilih berkas masukan maupun berkas keluaran
2. Pengguna tidak memilih mode kompresi ataupun dekompresi
3. Pengguna tidak memilih algoritma kompresi maupun dekompresi yang akan digunakan
4. Pengguna akan mengkonfirmasi suatu eksekusi
5. Proses kompresi maupun dekompresi telah selesai



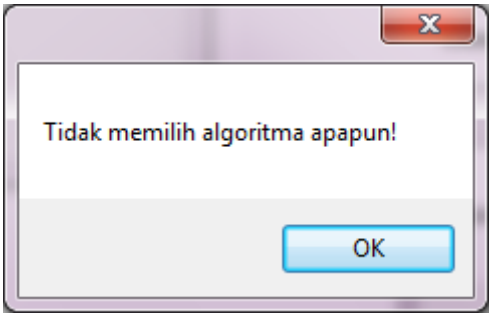
Gambar III-13: ***Open File Dialog***



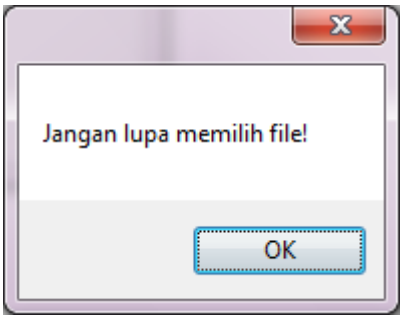
Gambar III-14: ***Save File Dialog***



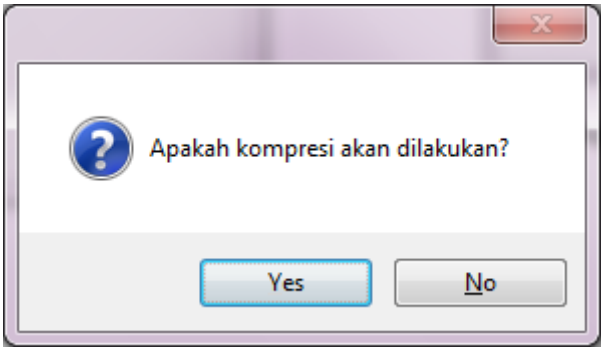
Gambar III-15: *Message Box* ketika tidak memilih mode apapun



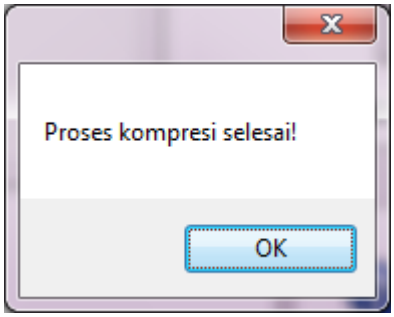
Gambar III-16: *Message Box* ketika tidak memilih algoritma satupun



Gambar III-17: *Message Box* ketika tidak memilih berkas



Gambar III-18: *Message Box* konfirmasi pengguna



Gambar III-19: *Message Box* ketika proses eksekusi selesai

BAB IV
EVALUASI

Pada bab ini, akan dijelaskan hal-hal yang meliputi evaluasi kinerja perangkat lunak. Hal-hal yang akan dijelaskan antara lain mengenai evaluasi yang dilakukan dan analisis hasil evaluasi. Evaluasi yang dilakukan antara lain evaluasi perangkat lunak dan evaluasi algoritma.

4.1 Evaluasi Perangkat Lunak

Evaluasi dilakukan untuk mengevaluasi kebenaran perangkat lunak yang mencakup proses kompresi dan dekompresi berkas. Pada evaluasi ini akan dilihat apakah perangkat lunak berhasil mengkompresi maupun mendekompresi berkas sehingga menjadi kembali ke berkas awal dengan sempurna.

4.1.1 Lingkungan Evaluasi

Evaluasi digunakan pada perangkat keras berbasis *desktop* dengan spesifikasi sebagai berikut:

1. Prosesor Intel Core i5 2,26 GHz
2. Memory RAM 2GB
3. Harddisk 500GB
4. Perangkat masukan *keyboard* dan *mouse*

4.1.2 Tujuan Evaluasi

Tujuan dari evaluasi perangkat lunak ini adalah:

1. Menguji kebenaran proses kompresi maupun dekompresi berkas dengan algoritma Lempel-Ziv-Welch
2. Menguji kebenaran proses kompresi maupun dekompresi berkas dengan berkas yang telah dimanipulasi dengan transformasi Burrows Wheeler

4.1.3 Data Uji

Data uji yang digunakan ialah berupa berkas teks yang terdiri atas berkas-berkas sebagai berikut.

Tabel IV-1: Data Uji Perangkat Lunak

No.	Nama Berkas	Deskripsi	Ukuran (bytes)
1	red_dead_redemption_c.txt	Panduan game “Red Dead Redemption”	335064
2	1musk10.txt	E-book berjudul: “The Three Musketeers”	1344739
3	anne11.txt	E-book berjudul: “Anne of Green Gables”	587053

4.1.4 Kasus Uji dan Hasil Evaluasi

Berdasarkan tujuan evaluasi yang telah dijabarkan pada subbab 4.1.2., maka didefinisikan dua buah kasus uji untuk mengevaluasi kebenaran perangkat lunak.

1. Kasus uji pertama: Menguji kebenaran proses kompresi dan dekompresi berkas dengan algoritma LZW. Pengujian dinyatakan diterima apabila berkas hasil kompresi dapat dikompresi dan tidak memiliki perbedaan dengan berkas awal sebelum dikompresi. Perbandingan berkas dilakukan dengan kakas Notepad++ dengan *Plugin Compare*.

Tabel IV-2: Hasil Pengujian Kasus Pertama

Berkas Masukan	Berkas Keluaran	Ukuran Masukan (byte)	Ukuran Keluaran (byte)	Binary Compare	Kesimpulan
red_dead_redemption_c.txt	red_dead_redemption_c(com).txt	335064	198436	Files Match!	Diterima
red_dead_redemption_c(com).txt	red_dead_redemption_c(decom).txt	198436	335064		Diterima
1musk10.txt	1musk10(com).txt	1344739	832923	Files Match!	Diterima
1musk10(com).txt	1musk10(decom).txt	832923	1344739		Diterima

Berkas Masukan	Berkas Keluaran	Ukuran Masukan (byte)	Ukuran Keluaran (byte)	Binary Compare	Kesimpulan
anne11.txt	anne11(com).txt	587053	368089	Files Match!	Diterima
anne11(com).txt	anne11(decom).txt	368089	587053		Diterima

2. Kasus uji kedua: Menguji kebenaran proses kompresi dan dekompresi berkas dengan algoritma LZW, dimana sebelum dikompresi berkas telah ditransformasikan dengan algoritma Burrows Wheeler. Pengujian dinyatakan diterima apabila berkas hasil kompresi dapat dikompresi dan setelah mengalami detransformasi tidak memiliki perbedaan dengan berkas awal sebelum dikompresi. Perbandingan berkas dilakukan dengan kakas Notepad++ dengan *Plugin Compare*.

Tabel IV-3: Hasil Pengujian Kasus Kedua

Berkas Masukan	Berkas Keluaran	Ukuran Masukan (byte)	Ukuran Keluaran (byte)	Binary Compare	Kesimpulan
red_dead_redemption_c.txt	red_dead_redemption_c(com).txt	335064	164238	Files Match!	Diterima
red_dead_redemption_c(com).txt	red_dead_redemption_c(decom).txt	164238	335064		Diterima
1musk10.txt	1musk10(com).txt	1344739	750028	Files Match!	Diterima
1musk10(com).txt	1musk10(decom).txt	750028	1344739		Diterima
anne11.txt	anne11(com).txt	587053	331543	Files Match!	Diterima
anne11(com).txt	anne11(decom).txt	331543	587053		Diterima

4.2 Evaluasi Algoritma

Evaluasi algoritma dilakukan untuk melakukan pengukuran terhadap kinerja algoritma Lempel Ziv Welch sebelum maupun sesudah dioptimisasi dengan transformasi Burrows Wheeler.

4.2.1 Lingkungan Evaluasi

Evaluasi digunakan pada perangkat keras berbasis *desktop* dengan spesifikasi sebagai berikut:

1. Prosesor Intel Core i5 2,26 GHz
2. Memory RAM 2GB
3. Harddisk 500GB
4. Perangkat masukan *keyboard* dan *mouse*

4.2.2 Tujuan Evaluasi

Tujuan evaluasi adalah untuk mengetahui perbandingan kinerja antara algoritma Lempel Ziv Welch dan algoritma Lempel Ziv Welch yang dioptimisasi dengan transformasi Burrows Wheeler. Kinerja yang akan diukur adalah rasio kompresi. Perubahan kinerja dihitung dengan persamaan berikut:

Kenaikan kinerja didefinisikan sebagai K_k:

$$K_k = ((Rasio\ LZW) - (Rasio\ LZW + BWT)) * 100\%$$

IV.1

4.2.3 Data Evaluasi

Data yang digunakan pada evaluasi ini berjumlah 33 buah, yaitu:

1. 14 berkas *calgary corpus*
2. 6 berkas *canterbury corpus*
3. 10 berkas teks biasa
4. 3 berkas kasus ekstrim

Tabel IV-4: Data Uji Evaluasi Algoritma

No	Nama Berkas	Deskripsi	Ukuran (bytes)	Kategori
1	bib.txt	Salah satu berkas contoh *calgary corpus*	111261	*Calgary Corpus*
2	book1.txt	Salah satu berkas contoh *calgary corpus*	768771	*Calgary Corpus*
3	book2.txt	Salah satu berkas contoh *calgary corpus*	610856	*Calgary Corpus*
4	news.txt	Salah satu berkas contoh *calgary corpus*	377109	*Calgary Corpus*
5	paper1.txt	Salah satu berkas contoh *calgary corpus*	53161	*Calgary Corpus*
6	paper2.txt	Salah satu berkas contoh *calgary corpus*	82199	*Calgary Corpus*
7	paper3.txt	Salah satu berkas contoh *calgary corpus*	46526	*Calgary Corpus*
8	paper4.txt	Salah satu berkas contoh *calgary corpus*	13286	*Calgary Corpus*
9	paper5.txt	Salah satu berkas contoh *calgary corpus*	11954	*Calgary Corpus*
10	paper6.txt	Salah satu berkas contoh *calgary corpus*	38105	*Calgary Corpus*
11	progc.txt	Salah satu berkas contoh *calgary corpus*	39611	*Calgary Corpus*
12	progl.txt	Salah satu berkas contoh *calgary corpus*	71646	*Calgary Corpus*
13	progp.txt	Salah satu berkas contoh *calgary corpus*	49379	*Calgary Corpus*
14	trans.txt	Salah satu berkas contoh *calgary corpus*	93695	*Calgary Corpus*
15	alice29.txt	Salah satu berkas contoh *canterbury corpus*	148481	*Canterbury Corpus*
16	asyoulik.txt	Salah satu berkas contoh *canterbury corpus*	125179	*Canterbury Corpus*

No	Nama Berkas	Deskripsi	Ukuran (bytes)	Kategori
17	lcet10.txt	Salah satu berkas contoh *canterbury corpus*	419235	*Canterbury Corpus*
18	plrabn12.txt	Salah satu berkas contoh *canterbury corpus*	471162	*Canterbury Corpus*
19	bible.txt	Salah satu berkas contoh *canterbury corpus*	4047392	*Canterbury Corpus*
20	world192.txt	Salah satu berkas contoh *canterbury corpus*	2473400	*Canterbury Corpus*
21	final_fantasy_xiii.txt	Game guide Final Fantasy XIII	996882	Teks biasa
22	dead_space_e.txt	Game guide Dead Space	592016	Teks biasa
23	devil_may_cry_4_c.txt	Game guide Devil May Cry 4	678043	Teks biasa
24	red_dead_redemption_c.txt	Game guide Red Dead Redemption	335064	Teks biasa
25	kingdom hearts BBS.txt	Game guide Kingdom Hearts Birth By Sleep	595829	Teks biasa
26	ninja_gaiden_ii_d.txt	Game guide Ninja Gaiden 2	231009	Teks biasa
27	dynasty_warriors_5.txt	Game guide Dynasty Warriors 5	258152	Teks biasa
28	fable_ii.txt	Game guide Fable 2	156595	Teks biasa
29	1musk10.txt	E-book berjudul: “The Three Musketeers”	1344739	Teks biasa
30	anne11.txt	E-book berjudul: “Anne of Green Gables”	587053	Teks biasa
31	extrim1.txt	Berkas berisi perulangan “A” sebanyak 1000 kali	1000	Berkas ekstrim
32	extrim2.txt	Berkas berisi perulangan “AB” sebanyak 1000 kali	2000	Berkas ekstrim
33	extrim3.txt	Berkas berisi perulangan “ABC” sebanyak 1000 kali	3000	Berkas ekstrim

4.2.4 Kasus Uji dan Pengujian

Kompresi berkas dengan algoritma kompresi LZW

Tabel IV-5: Hasil Pengujian dengan Algoritma LZW

No	Nama berkas	Ukuran awal (bytes)	Ukuran akhir (bytes)	Rasio LZW
1	bib.txt	111261	88855	0,798618
2	book1.txt	768771	540084	0,702529
3	book2.txt	610856	354867	0,580934
4	news.txt	377109	276258	0,732568
5	paper1.txt	53161	31607	0,594552
6	paper2.txt	82199	50269	0,611552
7	paper3.txt	46526	31152	0,669561
8	paper4.txt	13286	7425	0,558859
9	paper5.txt	11954	6947	0,581144
10	paper6.txt	38105	21603	0,566933
11	progc.txt	39611	24911	0,628891
12	progl.txt	71646	35456	0,494878
13	progp.txt	49379	22283	0,451265
14	trans.txt	93695	51405	0,548642
15	alice29.txt	148481	89134	0,600306
16	asyoulik.txt	125179	83666	0,668371
17	lcet10.txt	419235	254171	0,606273
18	plrabn12.txt	471162	334539	0,71003
19	bible.txt	4047392	2095318	0,517696
20	world192.txt	2473400	1292321	0,522488
21	final_fantasy_xiii.txt	996882	306439	0,307397
22	dead_space_e.txt	592016	302571	0,511086
23	devil_may_cry_4_c.txt	678043	359222	0,529792
24	red_dead_redemption_c.txt	335064	198436	0,592233
25	kingdom hearts BBS.txt	595829	307946	0,516836
26	ninja_gaiden_ii_d.txt	231009	125262	0,542239
27	dynasty_warriors_5.txt	258152	126884	0,491509
28	fable_ii.txt	156595	79119	0,505246

No	Nama berkas	Ukuran awal (bytes)	Ukuran akhir (bytes)	Rasio LZW
29	1musk10.txt	1344739	832923	0,619394
30	anne11.txt	587053	368089	0,627012
31	extrim1.txt	1000	78	0,078
32	extrim2.txt	2000	145	0,0725
33	extrim3.txt	3000	213	0,071

Kompresi berkas dengan algoritma kompresi LZW + BWT

Tabel IV-6: Hasil Pengujian dengan Algoritma LZW yang Dioptimisasi

No	Nama berkas	Ukuran awal (bytes)	Ukuran akhir (bytes)	Rasio LZW + BWT
1	bib.txt	111261	53856	0,484051
2	book1.txt	768771	390813	0,508361
3	book2.txt	610856	346549	0,567317
4	news.txt	377109	232814	0,617365
5	paper1.txt	53161	31203	0,586953
6	paper2.txt	82199	41679	0,50705
7	paper3.txt	46526	23926	0,51425
8	paper4.txt	13286	7445	0,560364
9	paper5.txt	11954	7023	0,587502
10	paper6.txt	38105	23305	0,6116
11	progc.txt	39611	24472	0,617808
12	progl.txt	71646	34918	0,487368
13	progp.txt	49379	23298	0,47182
14	trans.txt	93695	50547	0,539484
15	alice29.txt	148481	71775	0,483395
16	asyoulik.txt	125179	62976	0,503088
17	lcet10.txt	419235	221023	0,527206
18	plrabn12.txt	471162	232527	0,493518
19	bible.txt	4047392	1851272	0,457399
20	world192.txt	2473400	1569263	0,634456
21	final_fantasy_xiii.txt	996882	443313	0,4447
22	dead_space_e.txt	592016	297555	0,502613
23	devil_may_cry_4_c.txt	678043	346096	0,510434

No	Nama berkas	Ukuran awal (bytes)	Ukuran akhir (bytes)	Rasio LZW + BWT
24	red_dead_redemption_c.txt	335064	164238	0,490169
25	kingdom hearts BBS.txt	595829	238702	0,400622
26	ninja_gaiden_ii_d.txt	231009	114945	0,497578
27	dynasty_warriors_5.txt	258152	133200	0,515975
28	fable_ii.txt	156595	76350	0,487563
29	1musk10.txt	1344739	750028	0,55775
30	anne11.txt	587053	331543	0,564758
31	extrim1.txt	1000	72	0,072
32	extrim2.txt	2000	138	0,069
33	extrim3.txt	3000	204	0,068

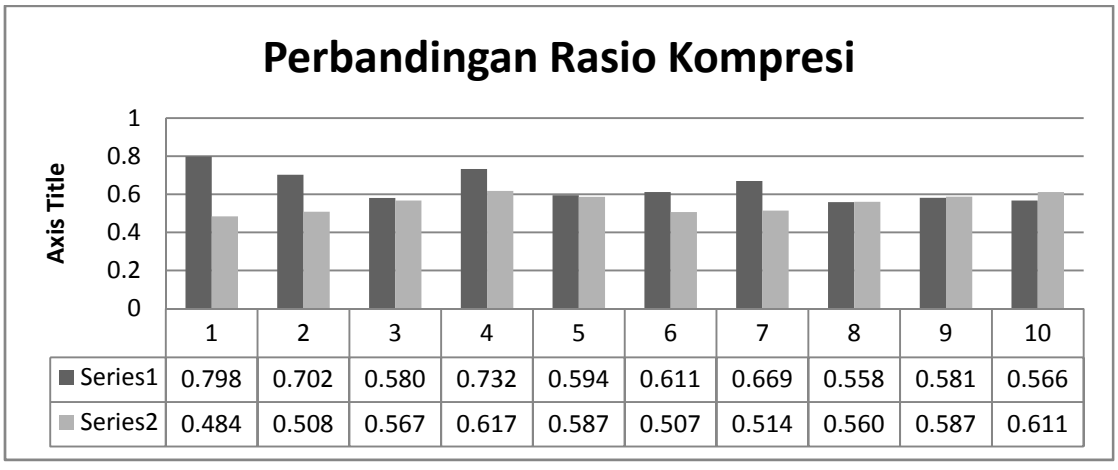
Nilai kenaikan kerja dapat dilihat pada tabel IV-7.

Tabel IV-7: Hasil Perhitungan Kenaikan Kerja

No	Nama berkas	K_k (%)
1	bib.txt	31,4567
2	book1.txt	19,4168
3	book2.txt	1,3617
4	news.txt	11,5203
5	paper1.txt	0,7599
6	paper2.txt	10,4502
7	paper3.txt	15,5311
8	paper4.txt	-0,1505
9	paper5.txt	-0,6358
10	paper6.txt	-4,4667
11	progc.txt	1,1083
12	progl.txt	0,751
13	progp.txt	-2,0555
14	trans.txt	0,9158
15	alice29.txt	11,6911
16	asyoulik.txt	16,5283
17	lcet10.txt	7,9067
18	plrabn12.txt	21,6512
19	bible.txt	6,0297

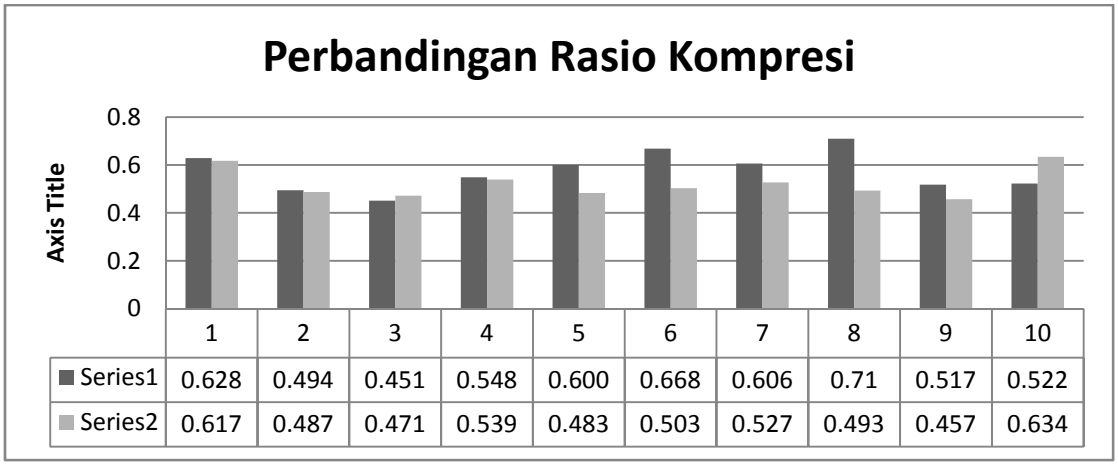
No	Nama berkas	K_k (%)
20	world192.txt	-11,1968
21	final_fantasy_xiii.txt	-13,7303
22	dead_space_e.txt	0,8473
23	devil_may_cry_4_c.txt	1,9358
24	red_dead_redemption_c.txt	10,2064
25	kingdom hearts BBS.txt	11,6214
26	ninja_gaiden_ii_d.txt	4,4661
27	dynasty_warriors_5.txt	-2,4466
28	fable_ii.txt	1,7683
29	1musk10.txt	6,1644
30	anne11.txt	6,2254
31	extrim1.txt	0,6
32	extrim2.txt	0,35
33	extrim3.txt	0,3

Perbandingan rasio kompresi pada berkas nomor 1-10 dapat dilihat pada gambar IV-1. *Series1* menunjukkan rasio kompresi berkas yang belum dioptimisasi, sedangkan *series2* menunjukkan rasio kompresi berkas yang telah dioptimisasi. Adapun urutan berkas sesuai dengan tabel IV-4.



Gambar IV-1: Perbandingan Rasio Kompresi Berkas 1-10

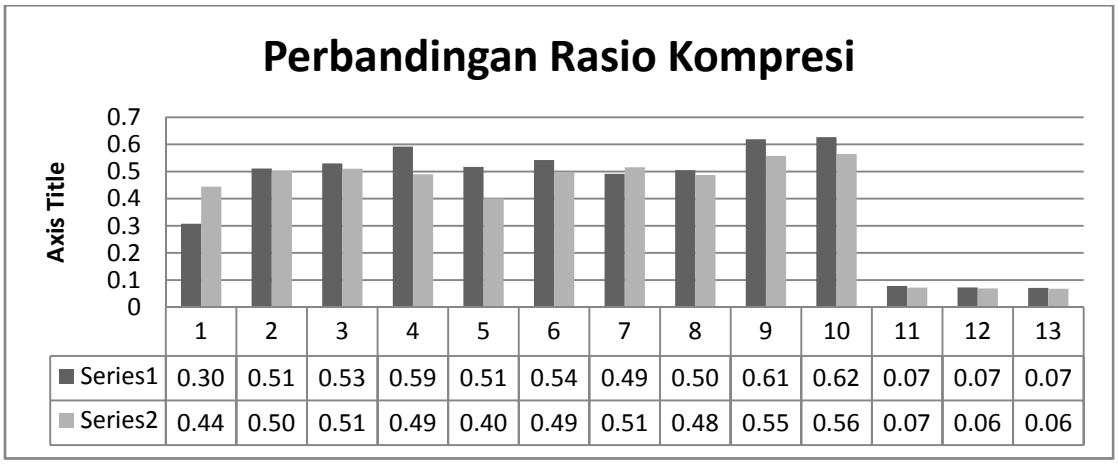
Perbandingan rasio kompresi pada berkas nomor 11-20 dapat dilihat pada gambar IV-2. *Series1* menunjukkan rasio kompresi berkas yang belum dioptimisasi, sedangkan *series2* menunjukkan rasio kompresi berkas yang telah dioptimisasi.



	1	2	3	4	5	6	7	8	9	10
Series1	0.628	0.494	0.451	0.548	0.600	0.668	0.606	0.71	0.517	0.522
Series2	0.617	0.487	0.471	0.539	0.483	0.503	0.527	0.493	0.457	0.634

Gambar IV-2: Perbandingan Rasio Kompresi Berkas 11-20

Perbandingan rasio kompresi pada berkas nomor 21-33 dapat dilihat pada gambar IV-3. *Series1* menunjukkan rasio kompresi berkas yang belum dioptimisasi, sedangkan *series2* menunjukkan rasio kompresi berkas yang telah dioptimisasi.



	1	2	3	4	5	6	7	8	9	10	11	12	13
Series1	0.30	0.51	0.53	0.59	0.51	0.54	0.49	0.50	0.61	0.62	0.07	0.07	0.07
Series2	0.44	0.50	0.51	0.49	0.40	0.49	0.51	0.48	0.55	0.56	0.07	0.06	0.06

Gambar IV-3: Perbandingan Rasio Kompresi Berkas 21-33

4.2.5 Analisis Hasil Pengujian

Berdasarkan evaluasi algoritma yang telah dilakukan, terdapat hal-hal yang dapat menjadi analisis hasil pengujian.

1. Untuk berkas dengan kategori *calgary corpus*:

Secara umum terdapat peningkatan kinerja pada algoritma kompresi LZW yang dioptimisasi dengan transformasi Burrows Wheeler. Peningkatan kinerja berkisar antara 0,7-30%. Namun terdapat 4 buah berkas yang mengalami penurunan kinerja, namun dalam jumlah yang tidak signifikan yaitu antara 0,1-4%.

2. Untuk berkas dengan kategori *canterbury corpus:*

Secara umum terdapat peningkatan kinerja pada algoritma kompresi LZW yang dioptimisasi dengan transformasi Burrows Wheeler. Peningkatan kinerja berkisar antara 6-21%. Namun terdapat sebuah berkas yang mengalami penurunan kinerjasebesar 11%.

3. Untuk berkas dengan kategori teks biasa:

Secara umum terdapat peningkatan kinerja pada algoritma kompresi LZW yang dioptimisasi dengan transformasi Burrows Wheeler. Peningkatan kinerja berkisar antara 0,8-11%. Namun terdapat 2 berkas yang mengalami penurunan kinerja dengan angka antara 2-13%.

4. Untuk berkas dengan kategori berkas ekstrim:

Secara umum terjadi peningkatan kinerja pada algoritma kompresi LZW yang dioptimisasi dengan transformasi Burrows Wheeler, namun dengan angka yang tidak begitu signifikan antara 0,3-0,6%.

Apabila dilihat secara keseluruhan, algorima kompresi LZW sebelum maupun sesudah dioptimisasi dengan transformasi Burrows Wheeler menghasilkan rasio kompresi yang cukup baik, yaitu sekitar 0,4-0,6. Hasil yang lebih baik lagi yaitu rasio kompresi pada berkas teks ekstrim, dimana nilai rasio kompresi dapat mencapai nilai 0,05.

Hasil dari pemampatan algoritma kompresi LZW sebelum dan sesudah dioptimisasi akan menghasilkan berkas dengan ukuran yang lebih kecil dibandingkan ukuran sebelum dikompresi. Hal ini menunjukkan bahwa algoritma LZW benar-benar dapat diandalkan untuk mengkompresi suatu berkas, karena tidak ada hasil evaluasi yang menyebabkan ukuran berkas menjadi lebih besar dari sebelumnya.

Kesimpulan dari pengujian ini ialah, secara umum transformasi Burrows Wheeler dapat mengoptimisasi algoritma kompresi LZW, namun sesungguhnya rasio kompresi dari algoritma LZW tersebut tanpa dioptimisasi sudah cukup baik, sehingga algoritma LZW dapat diandalkan dalam mengkompresi berkas-berkas yang berbentuk *plain teks*.

BAB V
KESIMPULAN DAN SARAN

Bab ini berisi beberapa kesimpulan dan saran yang diperoleh dari penelitian tugas akhir ini.

5.1 Kesimpulan

Kesimpulan yang didapatkan dari penelitian ini adalah sebagai berikut:

1. Secara umum, algoritma kompresi LZW yang dioptimisasi dengan transformasi Burrows Wheeler akan memberikan peningkatan kinerja untuk seluruh kategori berkas teks yang telah diujicobakan, karena transformasi Burrows Wheeler dapat mengubah berkas menjadi bentuk yang lebih mudah dimampatkan. Optimisasi yang dimaksud adalah transformasi Burrows Wheeler menjadi preproses dari berkas yang akan dikompresi.
2. Algoritma kompresi LZW sebelum maupun sesudah dioptimisasi dapat diandalkan dalam proses kompresi data karena nilai rasio kompresi yang dihasilkan berkisar antara 0,4-0,6. Dengan kata lain, berkas dapat terkompres dengan ukuran kurang lebih menjadi setengah kali lipat ukuran semula.

5.2 Saran

Saran yang dapat disampaikan untuk penelitian lebih lanjut adalah sebagai berikut:

1. Algoritma kompresi digunakan untuk berkas berjenis gambar, suara, maupun video. Untuk selanjutnya, bentuk berkas sebagai *plain text* dihilangkan dari pembatasan masalah.
2. Optimisasi algoritma kompresi digunakan dengan metode transformasi lain seperti Star dan LIPT. Namun, perlu dipertimbangkan bagaimana mengimplementasikan kamus tambahan pada algoritma transformasi tersebut.
3. Penghitungan kinerja algoritma selain memperhitungkan sisi rasio kompresi, juga bisa dengan memperhitungkan faktor waktu dan nilai entropi berkas.

DAFTAR REFERENSI

Fischer, Gerrard. (2001). *The Software Technology of the 21st Century: From Software Reuse to Collaborative Software Design*. University of Colorado.

Hankerson, Darrel. (2003). *Introduction to Information Theory and Data Compression 2nd Edition*. CRC.

Radescu, Radu. (2009). *Transform Methods Used In Lossless Compression of Text Files*. University of Bucharest.

Salomon, David. (2007). *Data Compression: The Complete Reference 4th Edition*. Springer-Verlag.

Shannon, C.E. (1948). *A Mathematical Theory of Communication*. Bell System Technical Journal vol 27.

Simpson, Jamie. (2008). *Words with Simple Burrows-Wheeler Transform*. Curtin University of Technology.

Witten, Ian H. (1999). *Managing Gigabyte: Compressing and Indexing Document and Images*. Kaufmann Publisher.

Ziv, Lempel. (1997). *A Universal Algorithm For Sequential data Compression*. IEEE.